

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

Definición y desarrollo
de Solución Informática
para el aprendizaje del
lenguaje musical

Curso: 2025/2026

Alumno/a:

José Luis Garrido Castro

Director/es:

Javier Criado Rodríguez

AGRADECIMIENTOS

Me gustaría dar las gracias, en primer lugar, a la Escuela de Música de Huércal de Almería, al Real Conservatorio Profesional de Música de Almería y la Escuela de Música Manuel Sánchez Alonso de Canjáyar por formarme durante mis 17 últimos años, poner en mi vida este maravilloso arte que es la música y por prestarme su valiosa ayuda para este proyecto.

En segundo lugar, a mi familia ya que siempre me han apoyado a lo largo de toda mi vida y se que se sienten muy orgullosos de mis logros, en especial, los conseguidos a lo largo de este grado de Ingeniería Informática. Me han dado todo y no me cabe duda que siempre me lo darán.

Después, cabe mencionar a mis amigos que siempre estaban ahí para animarme en momentos malos y sacarme de casa cuando me hacía falta.

En particular, quería destacar a mi compañero Jesús, con el cual llevo coincidiendo desde el instituto, manteniendo una estrecha amistad que se ha fortalecido durante el grado pasando grandes momentos y ayudándonos de forma recíproca en lo que hemos necesitado cada uno.

A mi pareja que hasta hoy no ha quedado día que no me haya preguntado como me iba con todos mis proyectos (como este) y que siempre consigue mandarme un empujón y sacarme una sonrisa para seguir llevando todo a cabo.

Por último, a todos los profesores que me han ayudado a lo largo de todo el grado y que me han transmitido todos los conocimientos posibles así como permitirme desarrollar la gran cantidad de nuevas habilidades que he conseguido durante estos años.

ÍNDICE GENERAL

	Página
1. Introducción	1
2. Objetivos	2
3. Marco teórico	3
3.1. Contexto	3
3.2. Relación entre la informática y el lenguaje musical	4
4. Investigación	5
4.1. Introducción	5
4.2. Análisis y comparación de las respuestas de los profesores	6
4.3. Análisis y comparación de las respuestas de los alumnos	7
4.4. Herramientas actuales	9
4.5. Necesidades detectadas	10
5. Estudio bibliográfico	11
5.1. Introducción	11
5.2. Mapping Literature Review	11
5.2.1. Definición la cadena de búsqueda y criterios	11
5.2.2. Selección de documentación relevante	12
5.2.3. Resultados obtenidos	14
5.2.3.1. El lenguaje musical a través de las TIC en educación primaria	14
5.2.3.2. Music Computer Technologies and Interactive Systems of Education in Digital Age School	15
5.2.3.3. Aplicación informática para la introducción del lenguaje musical	15
5.2.3.4. Online Gaming to Learn Music and English Language in Music and Ballet School Solfeggio Education	15
5.2.3.5. Multimedia technologies as a tool for teaching supervision over the students' skills (within the course on "music theory training")	16
5.2.4. Conclusiones	16

6. Definición de la solución	17
6.1. Especificación formal de la solución	17
6.2. Objetivos funcionales del sistema	19
6.3. Diagrama UML de casos de uso del sistema	20
6.4. Definición del diagrama de casos de uso	20
6.4.1. Actores	20
6.4.2. Casos de uso	21
6.4.2.1. Definición general	21
6.4.2.2. Requisitos funcionales	24
6.5. Requisitos no funcionales	47
6.6. Tecnologías y arquitectura de la solución	50
6.6.1. Frontend	50
6.6.2. Backend	51
6.6.3. Base de datos	52
6.6.4. Servicios externos	52
6.6.5. Integración y despliegue continuo	53
6.6.6. Arquitectura global de la aplicación	53
7. Planificación	55
8. Desarrollo de la aplicación backend	58
8.1. Arquitectura y visión general del backend	58
8.2. Modelado de datos	59
8.3. Implementación de la lógica de negocio e integración	60
8.3.1. Lógica de negocio: creación y actualización de calificaciones generales	61
8.3.2. Integración con servicios externos	63
8.3.3. Seguridad: autenticación mediante JWT	64
8.3.4. Documentación de la API con Swagger	66
8.4. Testing	67
8.4.1. Estrategia de pruebas	67
8.4.2. Herramientas utilizadas	67
8.4.3. Ejemplo de prueba	68
8.5. Despliegue del backend	70
9. Desarrollo de la aplicación frontend	72
9.1. Arquitectura y visión general del frontend	72

9.2. Implementación	73
9.2.1. Estructura del proyecto	73
9.2.2. Páginas, componentes y controladores	74
9.2.3. Manejo de rutas y navegación	78
9.2.4. Gestión del estado	79
9.2.5. Integración con el backend	81
9.3. Diseño y experiencia de usuario (UX/UI)	82
9.3.1. Componentes Ionic y su papel en el diseño	82
9.3.2. Diseño responsive y adaptabilidad	83
9.3.3. Usabilidad y experiencia de usuario	83
9.3.4. Accesibilidad	84
9.3.5. Ejemplos visuales de diseño y experiencia	84
9.4. Testing	86
9.4.1. Estrategia de pruebas	86
9.4.2. Herramientas utilizadas	86
9.4.3. Ejemplo de prueba	86
9.5. Despliegue del frontend	89
9.5.1. Proceso de build y optimización	89
9.5.2. Hosting y distribución	90
10. Integración y Despliegue Continuo	91
10.1. Descripción general del pipeline	91
10.2. Archivo de configuración del pipeline	91
10.3. Ventajas del pipeline integrado	93
11. Conclusiones y líneas de trabajo futuro	95
11.1. Conclusiones	95
11.2. Líneas de trabajo futuro	96
11.2.1. Plan de implantación y despliegue en entornos reales	96
11.2.2. Extensión de funcionalidades e innovación	96
11.2.3. Acceso sin autenticación y apertura a usuarios externos	97
11.2.4. Creación de la aplicación móvil	97
12. Anexo	98
12.1. Repositorio del proyecto	98
12.2. Estructura del proyecto	98



12.2.1. Estructura del backend	99
12.2.2. Estructura del frontend	99
12.3. Capturas adicionales de la aplicación	99
12.4. Entornos y herramientas de desarrollo	101
12.4.1. Entorno de desarrollo	101
12.4.2. Gestión y visualización de la base de datos	101
12.4.3. Herramientas de apoyo adicionales	102

ÍNDICE DE FIGURAS

3.1. Elementos del lenguaje musical	3
4.1. Entrevistas	6
4.2. Encuesta al alumnado	7
4.3. Respuestas a las preguntas 1, 3 y 6	8
4.4. Respuestas a las pregunta 4	8
5.1. Flujograma de la Mapping Review	13
6.1. Diagrama UML de casos de uso del sistema	20
6.2. Tecnologías frontend: Ionic, Angular, Jasmine y Karma	51
6.3. Tecnologías backend: Node.js, Express.js, JWT, Swagger, Jest y Supertest	52
6.4. MongoDB	52
6.5. Servicios externos	52
6.6. Tecnologías de integración y despliegue continuo	53
6.7. Arquitectura global de la aplicación	53
7.1. Cronograma del proyecto	56
7.2. Planificación de las tareas principales en horas	57
8.1. Modelo conceptual de datos del sistema	60
8.2. Interfaz de la documentación generada por Swagger	66
8.3. Pipeline de integración continua: Backend	71
9.1. Vista de inicio de sesión en móvil y escritorio	84
9.2. Menú lateral y barra de pestañas.	85
9.3. Uso de IonModal	85
9.4. Vista del perfil de profesor	86
9.5. Proceso simplificado de despliegue en GitHub Pages.	90



12.1. Vista de cada rama de la aplicación	100
12.2. Vista de administración	100
12.3. Vista de mensajería	100
12.4. Vista de formulario para rellenar cuestionarios	100

ÍNDICE DE TABLAS

5.1. Documentación recogida	14
6.1. Actor 1	20
6.2. Actor 2	21
6.3. Actor 3	21
6.4. Actor 4	21
6.5. Caso de uso: Iniciar sesión	21
6.6. Caso de uso: Recuperar contraseña	21
6.7. Caso de uso: Crear profesor	22
6.8. Caso de uso: Crear alumno	22
6.9. Caso de uso: Gestionar grupos	22
6.10. Caso de uso: Enviar mensaje	22
6.11. Caso de uso: Gestionar material de cada rama	22
6.12. Caso de uso: Gestionar tareas	22
6.13. Caso de uso: Gestionar entregas	23
6.14. Caso de uso: Gestionar entregas	23
6.15. Caso de uso: Gestionar calificaciones del alumnado	23
6.16. Caso de uso: Ver mensajes	23
6.17. Caso de uso: Interactuar con el material de cada rama	23
6.18. Caso de uso: Revisar calificaciones	23
6.19. Caso de uso: Interactuar con tareas	24
6.20. Caso de uso: Interactuar con cuestionarios	24
6.21. Requisito funcional: RF-01 Iniciar sesión	24
6.22. Requisito funcional: RF-02 Recuperar contraseña	25
6.23. Requisito funcional: RF-03 Crear profesor	25
6.24. Requisito funcional: RF-04 Validación y generación automática de credenciales	26
6.25. Requisito funcional: RF-05 Crear alumno	27

6.26. Requisito funcional: RF-06 Crear grupo	27
6.27. Requisito funcional: RF-07 Listar grupos	28
6.28. Requisito funcional: RF-08 Seleccionar grupo	28
6.29. Requisito funcional: RF-09 Enviar mensaje	29
6.30. Requisito funcional: RF-10 Ver libro del curso	29
6.31. Requisito funcional: RF-11 Subir o reemplazar libro del curso	30
6.32. Requisito funcional: RF-12 Eliminar libro del curso	31
6.33. Requisito funcional: RF-13 Revisar tareas	31
6.34. Requisito funcional: RF-14 Crear tarea	32
6.35. Requisito funcional: RF-15 Modificar tarea	33
6.36. Requisito funcional: RF-16 Cerrar tarea	34
6.37. Requisito funcional: RF-17 Eliminar tarea	34
6.38. Requisito funcional: RF-18 Revisar cuestionarios	35
6.39. Requisito funcional: RF-19 Crear cuestionario	36
6.40. Requisito funcional: RF-20 Modificar cuestionario	37
6.41. Requisito funcional: RF-21 Cerrar cuestionario	38
6.42. Requisito funcional: RF-22 Eliminar cuestionario	38
6.43. Requisito funcional: RF-23 Revisar entregas	39
6.44. Requisito funcional: RF-24 Calificar entrega	40
6.45. Requisito funcional: RF-25 Revisar calificaciones de entregas	41
6.46. Requisito funcional: RF-26 Revisar calificaciones generales del alumno	42
6.47. Requisito funcional: RF-27 Asignar calificación general al alumno	43
6.48. Requisito funcional: RF-28 Ver mensajes	44
6.49. Requisito funcional: RF-29 Entregar tarea	45
6.50. Requisito funcional: RF-30 Rellenar y entregar cuestionario	46
6.51. Requisito funcional transversal: RFT-1 Envío de notificaciones del sistema	47
6.52. Requisito no funcional: RNF-01 Escalabilidad y mantenibilidad	47
6.53. Requisito no funcional: RNF-02 Usabilidad y accesibilidad	48
6.54. Requisito no funcional: RNF-03 Diseño visual atractivo y motivador	48
6.55. Requisito no funcional: RNF-04 Portabilidad	48
6.56. Requisito no funcional: RNF-05 Seguridad	49
6.57. Requisito no funcional: RNF-06 Consistencia pedagógica	49



6.58. Requisito no funcional: RNF-07 Adaptabilidad del contenido	50
--	----

1 INTRODUCCIÓN

El proyecto debe cubrir la necesidad de recursos software en el lenguaje musical, o antiguamente conocido como solfeo, siendo esta una asignatura primordial para las enseñanzas en cualquier centro educativo donde se aprenda a tocar cualquier instrumento mediante el proyecto "Definición y desarrollo de Solución Informática para el aprendizaje del lenguaje musical".

Para esto, se ha realizado una investigación en el marco musical con la premisa de averiguar necesidades de profesorado y alumnado y obtener ejemplos de cualquier herramienta informática ya creada para las enseñanzas de los centros mencionados, dicha investigación comenzará alrededor de los centros que enseñen esta asignatura mediante una serie de objetos (como encuestas o entrevistas) que serán de gran utilidad a la hora de realizar una mejor cobertura a nivel de software a través de este mismo proyecto o futuros proyectos que se basen en este estudio.

Tras ello, se continuará la investigación mediante un estudio bibliográfico, en el cual, se obtendrá más información vital para la solución requerida, de esta forma, se tendrán dos grandes bloques de información, los cuales, como complemento a los datos obtenidos, serán comparados mediante la percepción propia como antiguo alumno de la asignatura.

Gracias a todo este trabajo se podrá definir una solución informática que cubra las necesidades detectadas y que, posteriormente, será desarrollada.

2 OBJETIVOS

El objetivo principal es, por tanto, **resolver la problemática en relación con la falta de recursos informáticos enfocados a la enseñanza del lenguaje musical**. Para alcanzar dicho objetivo, se deben cumplir, a su vez, los siguientes subobjetivos:

- **Entender las necesidades de profesores y alumnos de lenguaje musical.**
- **Aprender sobre la relación existente entre la informática y el lenguaje musical.**
- **Definir una solución informática que cubra las necesidades detectadas.**
- **Desarrollar la solución.**
- **Llevar a cabo el proceso de desarrollo con tecnologías totalmente gratuitas.**

3 MARCO TEÓRICO

3.1 CONTEXTO

El lenguaje musical, como se ha mencionado, es una asignatura enseñada en la gran mayoría de centros de estudio de la música, pero, como su propio nombre indica, también es un lenguaje o "idioma" (es por ello que la asignatura en la que se estudia tiene el mismo nombre).

Al igual que ocurre con otros idiomas, tanto en la comunicación escrita como oral, es el método que se usa para transmitir algo mediante la música. Por tanto, al igual que existen las letras, las palabras o los distintos signos de puntuación, en este caso, se tendrán una serie de elementos para establecer la comunicación [1]:



Figura 3.1: Elementos del lenguaje musical

Para aprender a utilizar todos estos elementos tanto de forma escrita como oral es necesario estudiar las siguientes cuatro ramas:

- **Ritmo:** Es la parte del lenguaje musical que consiste en la enseñanza de la lectura de partituras con fluidez.
- **Entonación:** Esta rama se encarga del estudio de la entonación de partituras con fluidez.
- **Audición:** La audición es otra de las ramas del lenguaje musical y en ella se entrena el oído para ser capaz de entender cualquier fragmento musical con el fin de poder plasmarlo (principalmente de forma escrita).
- **Teoría:** Por último, existe una parte en el lenguaje musical que consiste en la enseñanza teórica de la misma.

A través del estudio de las anteriores, se puede llegar a conseguir un dominio completo en el uso del lenguaje musical, por lo tanto, es importante tenerlo en cuenta de cara a la construcción de una solución.

Otro aspecto fundamental es la diferenciación entre escuelas de música y conservatorios. Los primeros, habitualmente, están gestionados por asociaciones y/o pequeñas empresas y, aunque pueden beneficiarse de alguna subvención local, son por tanto privados, lo que significa que cada uno puede tener una manera muy diferente de enfocar la asignatura por lo que las necesidades pueden ser distintas, en cambio, los conservatorios son públicos, de esta forma, la manera de enseñar la asignatura será muy parecida entre todos los centros.

3.2 RELACIÓN ENTRE LA INFORMÁTICA Y EL LENGUAJE MUSICAL

Para entender el dominio del proyecto es primordial entender su relación con la informática, en este caso, con el objetivo de definir y construir una solución. A priori, puede parecer que el lenguaje musical y la informática sean dos ámbitos bastante alejados, pero en el primero se ha detectado un grave problema y es el método de enseñanza que tiene mediante lecciones magistrales que provocan una sensación de aburrimiento en el alumnado que, a su vez, conlleva a una gran falta de interés por parte de los mismos y que desemboca en un aprendizaje lento. Es por ello, que la ingeniería podría tener un buen impacto al ser su principal objetivo resolver los problemas, en este caso, a través de la informática.

4 INVESTIGACIÓN

4.1 INTRODUCCIÓN

Como se ha mencionado, se ha llevado a cabo una investigación alrededor de los centros de enseñanza musical, para ello, se han diseñado una serie de encuestas y entrevistas enfocadas tanto al profesorado como al alumnado de cara a entender sus necesidades. Lo primero que se ha tenido en cuenta son esas diferencias que pueda haber entre los centros públicos y privados, por lo que ha sido necesario visitar dos entornos diferentes, el Real Conservatorio Profesional de Música de Almería (público) y la Escuela de Música de Huércal de Almería (privado).

En ambos centros se ha tenido cita con un profesor de la asignatura para tener una entrevista y entregarles una encuesta para que más tarde fueran devueltas y rellenas por todo el alumnado al cargo de los profesores mencionados. Tras ello, han sido analizadas todas las respuestas y comparadas las del público con las del privado (tanto las entrevistas como las encuestas) que, como se podrá apreciar más tarde, las preguntas también han tenido ligeras modificaciones entre las cuestionadas en un centro y otro.

A través de este método, se podrán conocer las herramientas informáticas usadas actualmente, lo cual será de ayuda en el estudio bibliográfico y así poder poner el foco sobre las funcionalidades más interesantes.

Por último, se extraerán también del análisis las necesidades del profesorado y el alumnado actuales, lo cual servirá para poder definir correctamente las posibles soluciones a la problemática.

4.2 ANÁLISIS Y COMPARACIÓN DE LAS RESPUESTAS DE LOS PROFESORES

Las preguntas y respuestas realizadas en las entrevistas de ambos profesores han sido las siguientes:

- | | |
|--|---|
| <ol style="list-style-type: none"> 1. ¿Cómo se llama? Juan Vicente García García 2. ¿Qué edad tiene? 61 años 3. ¿Qué formación tiene? (Carreras, títulos, etcétera...) Grado superior de guitarra, grado superior de solfeo y teoría de la música, grado profesional de composición y orquestación y grado profesional de piano 4. ¿Se maneja bien con la informática? Lo justo para trabajar 5. ¿Cree que los recursos informáticos del conservatorio a nivel de Software son suficientes o considera necesaria la creación de más recursos? Hay falta porque, aunque se está sustituyendo el formato tradicional por el moderno se avanza a un ritmo lento. 6. ¿Utiliza los recursos informáticos mencionados con frecuencia en la asignatura? Sí. 7. ¿Es difícil para usted manejarlos? No, son sencillos. 8. ¿Ha invertido mucho tiempo en aprender a usarlos? No 9. ¿Considera oportuna la creación de más recursos informáticos para la asignatura? Sí, mientras sean herramientas útiles son necesarias. 10. ¿Haría un esfuerzo en aprender a usar el nuevo software si cree que merece la pena, o seguiría dando sus clases como lo hacía hasta ahora? Sí 11. ¿Conoces Duolingo o Moodle y lo has usado en alguna ocasión? Sí 12. ¿Qué utilidades como profesor cree que son necesarias en el software creado fijándonos, por ejemplo, en las aplicaciones mencionadas antes? Incluir alguna funcionalidad tipo "cuestionario" ya que el Software actual no lo incluye. 13. ¿Qué ramas considera que requieren de más apoyo informático? Ritmo y audición 14. ¿Por qué? Porque hay menos recursos dedicados a ellas 15. ¿Cree que se debería buscar la creación de un software bonito y atractivo para sus alumnos o mantener el estilo serio y profesional que caracteriza al ámbito académico? Atractivas sobre todo enfocándolo a los alumnos bastante pequeños que la asignatura suele tener 16. ¿Por qué? El alumnado está muy apático actualmente y necesita que la herramienta llame la atención. | <ol style="list-style-type: none"> 1. ¿Cómo se llama? Natalia Botella Galiana 2. ¿Qué edad tiene? 32 años 3. ¿Qué formación tiene? (Carreras, títulos, etcétera...) Grado de pedagogía musical del clarinete 4. ¿Se maneja bien con la informática? Sí. 5. ¿Cree que los recursos informáticos del centro a nivel de Software son suficientes o considera necesaria la creación de más recursos? Hay falta, sobre todo en la teoría y la audición existen carencias de recursos. 6. ¿Cómo se han obtenido los recursos de la escuela? ¿Han sido muy costosos a nivel de tiempo de instalación y dinero? La licencia del Software venía con la compra de los libros que eran bastante caros, en cambio, la instalación fue bastante sencilla. 7. ¿Cree que el organismo encargado de la educación de esta asignatura (Junta de Andalucía) debería ordenar subvenciones o encargarse de crear y obtener algunos recursos informáticos para las escuelas privadas como esta? Sí 8. ¿Utiliza los recursos informáticos mencionados con frecuencia en la asignatura? Sí 9. ¿Es difícil para usted manejarlos? No 10. ¿Ha invertido mucho tiempo en aprender a usarlos? No 11. ¿Considera oportuna la creación de más recursos informáticos para la asignatura? Sí 12. ¿Haría un esfuerzo en aprender a usar el nuevo software si cree que merece la pena, o seguiría dando sus clases como lo hacía hasta ahora? Sí 13. ¿Qué ramas considera que requieren de más apoyo informático? Entonación y audición. 14. ¿Por qué? Porque, aunque teoría tampoco está muy cubierta, la escuela enseña a alumnos en una etapa muy inicial en la cual es más complicado el estudio de la entonación y la audición, por lo que cree que es más necesario fijarse en estas partes. 15. ¿Cree que se debería buscar la creación de un software bonito y atractivo para sus alumnos o mantener el estilo serio y profesional que caracteriza al ámbito académico? Más atractivo enfocado a un alumnado de muy corta edad. 16. ¿Por qué? Porque les vendrá bien que el servicio les llame la atención. |
|--|---|

(a) Profesor del conservatorio

(b) Profesora de la escuela de música

Figura 4.1: Entrevistas

En primer lugar, se puede apreciar que se han cambiado dos preguntas entre entrevista, esto es debido a que en el ámbito privado era importante obtener información relativa a la obtención de los recursos (lo cual está más claro en lo público) y, por el contrario, era interesante preguntar sobre herramientas de la Junta como Moodle al profesor del Conservatorio (Duolingo era para dar un ejemplo diferente). Tras ver estas respuestas se pueden concluir los siguientes aspectos:

1. A pesar de la diferencia de edad, ambos coinciden en que tienen la habilidad y predisposición suficiente para aprender a usar un nuevo servicio informático siempre y cuando sea intuitivo debido a la escasez y necesidad de recursos en ambos sectores (público y privado).
2. Ambos le dan bastante uso a los recursos que tienen actualmente.

3. También coinciden que la audición está menos cubierta, en cambio, en lo público parece que en el ritmo existe una mayor necesidad y en lo privado en la entonación. Aquí es importante señalar que las escuelas de música solo se suelen encargar de alumnos en etapas iniciadas mientras en los conservatorios sí se profundiza más en la asignatura, por tanto, si los alumnos necesitan más ayuda en la entonación en una etapa tan primeriza es normal que se ponga el foco más en ese aspecto mientras que en unas enseñanzas más avanzadas quizás no sea tan necesario.
4. Por último, también están de acuerdo en que la herramienta debería ser atractiva. Hay que poner atención a que se mencionó la necesidad de una funcionalidad tipo **cuestionario**.

4.3 ANÁLISIS Y COMPARACIÓN DE LAS RESPUESTAS DE LOS ALUMNOS

La encuesta realizada para los alumnos ha sido la misma para ambos centros, ya que no se ha considerado que personas de una edad tan temprana puedan apreciar diferencias en función del centro y otro. Estas han sido las preguntas:

Encuesta al alumnado:

1. ¿Tiene un buen manejo con las herramientas informáticas que dispone su centro para el lenguaje musical (aplicaciones de móvil, programas de ordenador o páginas web que apoyen el aprendizaje de dicha asignatura)?
Sí ☐ No ☐
2. Del 1 al 10 ¿Qué grado de dificultad presenta el uso de estas aplicaciones? (Siendo 1 extremadamente fácil y 10 extremadamente difícil):
3. ¿Consideras necesario que el centro disponga de más herramientas informáticas para la asignatura mencionada?
Sí ☐ No ☐
4. Si ha respondido "sí" a la pregunta anterior, ¿Cuál o cuáles de las cuatro partes que se estudian cree que tienen una mayor falta de aplicaciones?
Ritmo ☐ Entonación ☐ Teoría ☐ Audición ☐
5. ¿Por qué cree que las opciones que ha marcado en la pregunta anterior requieren de apoyo informático?

6. ¿Cree que la herramienta informática debería verse colorida, bonita y atractiva? ¿O piensa que debería de ser más seria y profesional? (siendo esto último más propio en este ámbito académico)
Colorida y atractiva ☐ Seria y profesional ☐
7. ¿Por qué?

!!!!Gracias por responder a la encuesta!!!!

Figura 4.2: Encuesta al alumnado

Se exponen en una gráfica las respuestas a las preguntas 1, 3 y 6 en primer lugar, se debe tener en cuenta que el total de respuestas de cada pregunta de ambas gráficas varía un poco debido a que, en algunos casos, dicha respuesta estaba en blanco o no era adecuada:

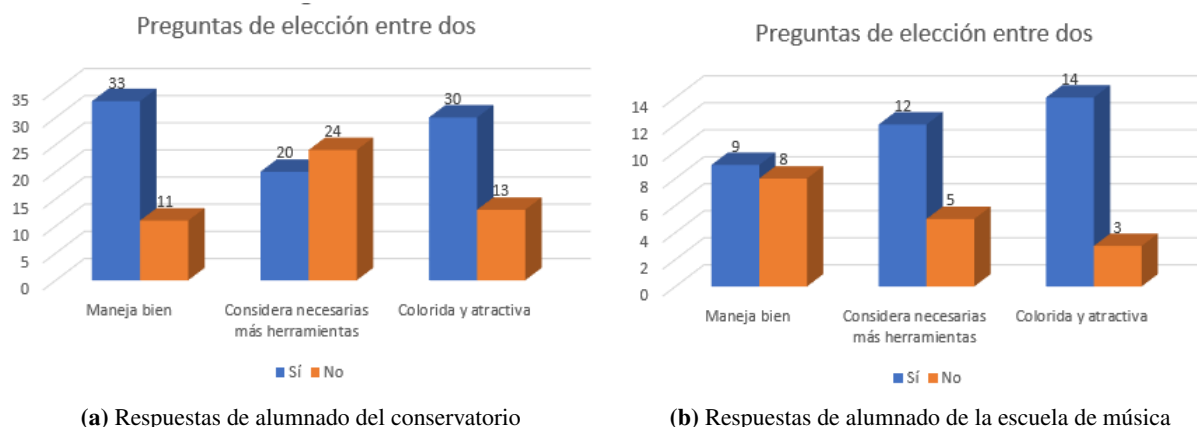


Figura 4.3: Respuestas a las preguntas 1, 3 y 6

Las respuestas de la pregunta 4 también han sido reflejadas en una gráfica:

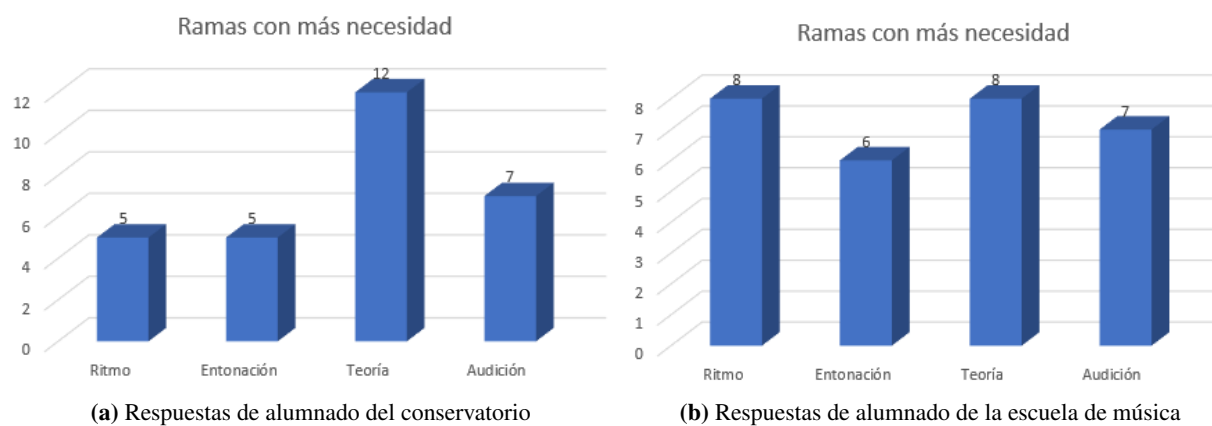


Figura 4.4: Respuestas a la pregunta 4

Cabe destacar que la media en la pregunta 2 ha sido de **4.68** para el conservatorio y de **2.58**.

Para las preguntas 5 y 7 se han seleccionado las mejores respuestas:

■ Mejores respuestas de la pregunta 5:

- (Marcó todas las opciones en la pregunta 4) "Ninguna tiene apoyo informático y es necesario poder acceder al material desde casa".
- (Marcó teoría en la pregunta 4) "Porque se debe poder acceder al contenido de clases anteriores para que no caiga en el olvido".
- (Marcó teoría en la pregunta 4) "Porque en las calificaciones de las actividades de teoría actualmente no se puede saber en qué te has equivocado".

■ Mejores respuestas de la pregunta 7:

- (Marcó ambas en la pregunta 6) "Creo que podría ser ambas cosas".
- (Marcó colorida y atractiva en la pregunta 6) "Al llamar más la atención se crea un ambiente más llevadero y también es importante disfrutar un poco de nuestras responsabilidades".

Por lo tanto, se sacan las siguientes conclusiones:

1. En la escuela de música están mucho más de acuerdo en querer más herramientas informáticas y, a su vez, les cuesta menos el uso de las herramientas actuales, muy posiblemente sea debido a que, en el conservatorio, el alumnado es generalmente mayor que en la escuela de música además de que, por experiencia personal, suele haber apuntados más adultos y es muy posible que estén menos dispuestos al uso de nuevos servicios.
2. Están bastante de acuerdo en que el software debe ser colorido y atractivo.
3. En contradicción con los profesores, creen más necesario reforzar los recursos informáticos dedicados a la teoría. También encuentran necesidad en la audición y el ritmo mientras que la entonación quedaría un peldaño por debajo.
4. Algunos aspectos referentes a la funcionalidad de software mencionados en las respuestas son la necesidad de poder acceder a material antiguo y desde casa y poder saber por qué se ha cometido un error en las correcciones de teoría.

4.4 HERRAMIENTAS ACTUALES

Ambos profesores se sirven de la aplicación web **EstudioMusica** [2] y se observa que las escuelas de música generalmente también usan herramientas incluidas con el material físico (libros de texto por ejemplo).



4.5 NECESIDADES DETECTADAS

Mezclando las necesidades de profesorado y alumnado, se llega a las siguientes conclusiones:

1. Es muy necesario apoyar a los centros de educación musical en cuanto a recursos informáticos debido a su escasez.
2. En general, profesorado y alumnado está a favor de incentivar la informática en las aulas.
3. Es de vital importancia que las aplicaciones sean coloridas y atractivas.
4. Las ramas más importantes son la auditiva por parte del profesorado y la teórica por parte del alumnado. También se ha mencionado que podría ser muy útil reforzar la rama de la entonación en las etapas iniciales del estudio de la asignatura.
5. Es necesario crear una funcionalidad tipo cuestionario, en la cual, se puedan añadir las correcciones y justificación de por qué está bien o mal un ejercicio. También podría ser útil que el servicio fuera accesible desde casa y el material antiguo se mantuviera en el mismo.

5 ESTUDIO BIBLIOGRÁFICO

5.1 INTRODUCCIÓN

El siguiente aspecto a tratar es la bibliografía. Una vez definidas unas conclusiones de la investigación previa, es necesario conocer qué documentación existe relacionada con el ámbito abarcado para extraer y sintetizar una información de interés de cara a la construcción de la solución. Por lo tanto, es necesario establecer unos pasos a seguir que permitan obtener la documentación necesaria, de manera que, se ha elegido hacer uso de la MLR (Mapping Literature Review), ya que establece una metodología marcada propia del dominio de la informática.

Los pasos a seguir serán los siguientes:

1. Definición de la pregunta de interés y los criterios de inclusión y exclusión de los estudios.
2. Localización y selección de la documentación relevante.
3. Síntesis de la información de la selección.
4. Análisis e interpretación de los resultados presentados.

5.2 MAPPING LITERATURE REVIEW

5.2.1 Definición la cadena de búsqueda y criterios

En primer lugar, se deben tener en cuenta los siguientes componentes clave:

- Población específica y contexto: Áreas de conocimiento del software enfocado a la enseñanza del lenguaje musical.
- Intervención: Funcionalidades y características interesantes desde el punto de vista pedagógico del lenguaje musical.
- Comparación: Otros software dedicados a la enseñanza del lenguaje musical
- Resultado: Necesidades y conclusiones para una solución.

Tras ello, se formulan una serie de preguntas en base a lo anterior:

1. ¿Qué softwares son más usados en la enseñanza del lenguaje musical?
2. ¿Qué características debe tener un software para que sea pedagógico desde el punto de vista musical?
3. ¿Qué características son más importantes en un software para la asignatura?
4. ¿Qué necesidades no están cubiertas por el software actual?

Una vez identificadas las preguntas, es necesario crear una cadena de búsqueda que se encargue de encontrar documentación dedicada tan solo a responderlas. La cadena sería la siguiente:

("software" OR "system" OR "application" OR "service") AND ("musical language teaching" OR "musical language education" OR "musical language pedagogy") AND ("requirements" OR "characteristics" OR "functionality")

Pero debido a que ninguna biblioteca daba resultados (excepto Google Scholar por ser muy genérica), se han tenido que disminuir en gran medida los términos de la cadena de búsqueda finalmente:

("software" OR "system" OR "application" OR "service") AND ("pedagogy" OR "teaching") AND "musical language"

Esto es consecuencia de que los temas tratados son muy dispares y es muy posible que exista muy poca documentación en la que se encuentren relacionados.

Por último, se han seleccionado los siguientes criterios de exclusión:

- No es accesible (documento privado o de pago).
- No abarca cualquiera de los dos grandes temas (informática y/o lenguaje musical) y/o no los relaciona en ningún punto.
- No está escrito en inglés o español.

5.2.2 Selección de documentación relevante

Los motores de búsqueda utilizados han sido: Google Scholar, Scopus, Association for Computing Machinery (ACM) Library y UVA doc, este último, porque al hacer búsquedas previas en crudo para analizar los campos, en la universidad de Valladolid (a la cual pertenece el repositorio mencionado), ya se habían hecho algunos TFGs similares. Para el motor de búsqueda de google se han analizado tan solo los primeros 50 resultados ya que, a partir de ahí, comenzaban a tener muy poca relevancia.



El flujograma ha sido el siguiente:

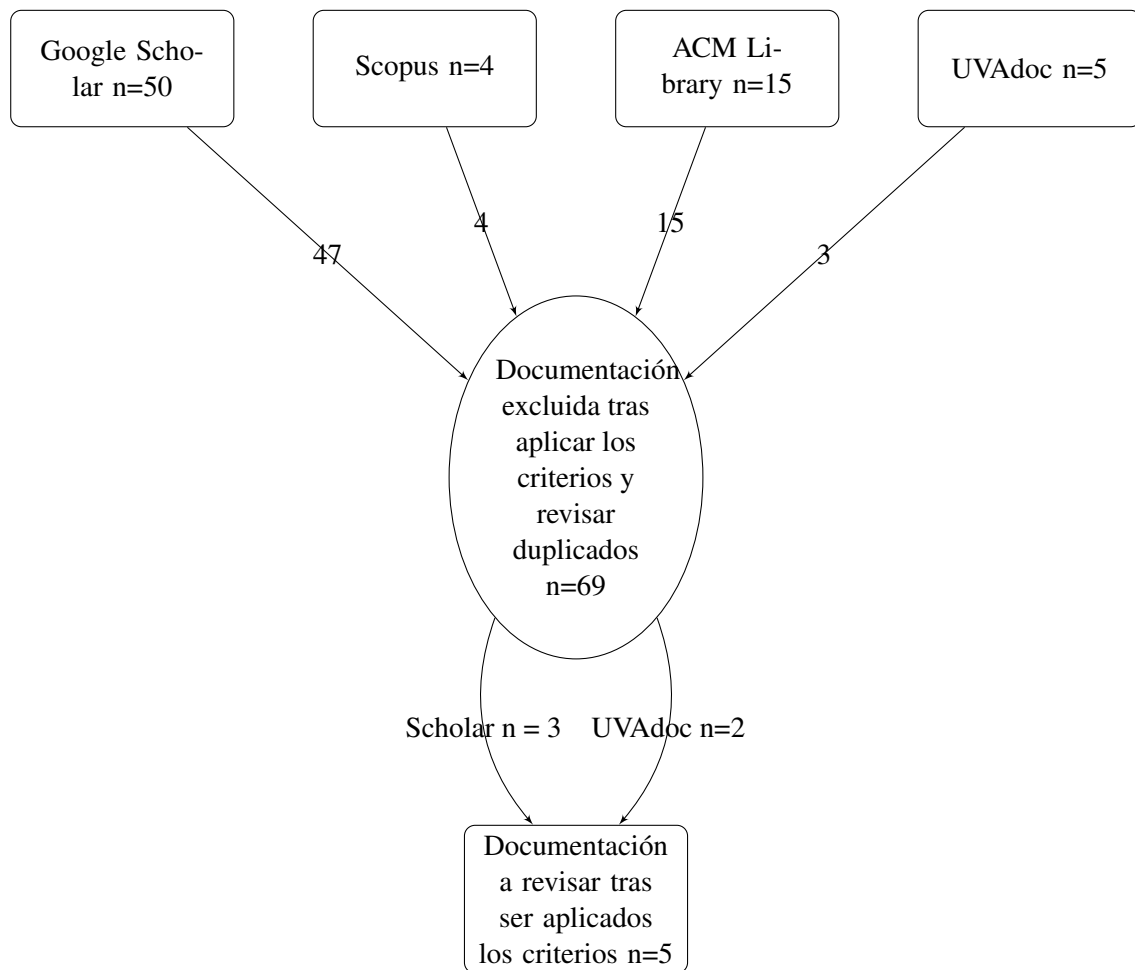


Figura 5.1: Flujograma de la Mapping Review

A pesar de la baja cantidad de documentos a revisar, abrir los términos de búsqueda para obtener más resultados no es una buena opción puesto que, el número de documentos irrelevantes ya es grande y solo se obtendrían aún más resultados innecesarios. Esto se debe a que, el marco teórico establecido que combina la informática con la música está bastante poco explorado aún.

Esta es la documentación obtenida (ordenada de por fecha de publicación descendente):

Fecha	Documentación recogida		
	Sitio de publicación	Tipo	Título
2022	UVAdoc	Trabajo técnico	El lenguaje musical a través de las TIC en educación primaria
2018	AtlantisPress	Artículo	Music Computer Technologies and Interactive Systems of Education in Digital Age School
2017	UVAdoc	Trabajo técnico	Aplicación informática para la introducción del lenguaje musical (Leemusica) en dispositivos móviles
2015	Hejmec Journal	Artículo	Online Gaming to Learn Music and English Language in Music and Ballet School Solfeggio Education
2014	Life Science Journal	Artículo	Multimedia technologies as a tool for teaching supervision over the students' skills (within the course on "music theory training")

Tabla 5.1: Documentación recogida

5.2.3 Resultados obtenidos

Como se ha descrito al principio de la revisión, el objetivo es encontrar soluciones o respuestas a una serie de problemáticas concretas que nos permitirán deducir una serie de conclusiones válidas para el proyecto. Para ello, se hará una breve síntesis de cada uno de los documentos enfocada en responder a las preguntas expuestas.

El lenguaje musical a través de las TIC en educación primaria

Este primer documento es un Trabajo de Fin de Grado de la Universidad de Valladolid que consiste en la creación de un blog que funcione como apoyo para la enseñanza del lenguaje musical y, por tanto, se apoya mucho a su vez, en otro tipo de herramientas centrando su proyecto en la innovación, el ámbito pedagógico y el método de aprendizaje que deben seguir los alumnos sobre todo teniendo en cuenta que son en su mayoría de edades muy tempranas. Las herramientas en las que hace énfasis y en las que se podría poner el foco son editores de partituras como MuseScore o Noteflight, editores de audio como Audacity y aplicaciones de encuestas o juegos como Kahoot o Quizziz, en consecuencia, se deduce que las funcionalidades más interesantes que expone son las que aportan dichas herramientas. También, al tocar la parte de la innovación, insiste en el punto de que es contradictorio el hecho de que la asignatura sea impartida desde un punto de vista tan teórico, puesto que, es muy importante para el músico tener una formación práctica siendo esta, además, una vía mucho más amena, por ello, es importante por la parte pedagógica que el alumno interactúe por su cuenta y tenga forma de trabajar los contenidos sin necesidad del estudio de memorización establecido actualmente. Por último, destaca que es importante llevar a cabo este proceso poco a poco, pues hay aún muchos profesores que están habituados a un sistema muy teórico y tradicional sin apoyarse demasiado en las TIC y que sería imposible realizar una

adaptación de golpe, debido a ello, es importante encontrar un balance entre las innovaciones y el sistema actual [3].

Music Computer Technologies and Interactive Systems of Education in Digital Age School

En segundo lugar, se encuentra un artículo a modo de análisis de la influencia y presencia de la tecnología en el aprendizaje de la música así como sus ventajas e inconvenientes. Además, se comenta sobre un nuevo software llamado "Soft Way to Mozart System", el cual, trata de paliar esas necesidades que actualmente se presentan a la hora de aprender a tocar un instrumento. Al parecer, en este caso, trata más del aprendizaje del instrumento en sí que del lenguaje musical, aún así se nos aporta la información de que, mediante el software, existe una posibilidad muy grande de apoyar la enseñanza de la música en general. Además, cabe destacar, que una de las claves es que los alumnos, al ser de edades tempranas, es muy importante que puedan interactuar con las herramientas y no sea el profesor el único que lo haga [4].

Aplicación informática para la introducción del lenguaje musical

En el siguiente documento, se presenta otro Trabajo de Fin de Grado de la Universidad de Valladolid que consiste en el desarrollo de una aplicación para la enseñanza del lenguaje musical (más enfocado al desarrollo que el anterior) en edades tempranas como educación infantil. Vuelve a nombrar una serie de herramientas compatibles con el aprendizaje de la asignatura como MuseScore, Audacity o creadores de encuestas/actividades e incluso algunas nuevas como Hydrogen que sirve para crear patrones de percusión con los que trabajar el ritmo con los alumnos y Band in a Box para patrones melódicos. En este caso, el objetivo de la herramienta es que los alumnos sean partícipes de la herramienta puesto que, para alumnos de educación infantil, los recursos creados hasta ahora los tiene que usar el profesor debido a su dificultad. Para ello, centra mucho su aplicación en la pedagogía a través del diseño, no solo haciéndolo llamativo, sino también usando colores para diferenciar notas entre otros recursos psicológicos. Por último, vuelve a hacerse énfasis en que la innovación no debe llegar de forma brusca, en este caso, debido a que la tecnología es un recurso complementario y no puede condicionar los contenidos que se dan en la asignatura, así como quitar demasiado tiempo al profesor para la enseñanza de los mismos [5].

Online Gaming to Learn Music and English Language in Music and Ballet School Solfeggio Education

A continuación, se trata un artículo que se centra en el enriquecimiento que ganan los alumnos de música y otras artes en general al jugar videojuegos exponiendo que, a día de hoy, es muy importante desarrollar más recursos TIC enfocados en la música. El documento aporta una serie de páginas web con juegos on-line musicales que se pueden usar como software de referencia, estas son "Classics for Kids Games" y "New York Philharmonic Kidzone" [6].

Multimedia technologies as a tool for teaching supervision over the students' skills (within the course on "music theory training")

Por último, se revisa este artículo que consiste en la búsqueda de métodos para poder evaluar las habilidades del alumno de música, y parece ser que el mejor para ello es las tecnologías multimedia. También incide en las competencias a evaluar y la metodología. Para el proyecto actual, tan solo es importante tomar en cuenta lo mencionado al principio sobre que es muy importante establecer la posibilidad de que se pueda llevar una supervisión y evaluación del alumno mediante los recursos TIC [7].

5.2.4 Conclusiones

Tras analizar los diferentes documentos se sacan las siguientes conclusiones:

1. Los software que se recomiendan, en general, son herramientas de apoyo que no pueden abarcar el grueso del temario de la asignatura y, además, son recursos muy completos que no pueden ser introducidos en este trabajo ya que por sí solos son un proyecto muy completo (línea de trabajo futura), exceptuando los juegos y cuestionarios en los que sí podría ser interesante poner el foco.
2. Se insiste mucho en la pedagogía en todos los documentos ya que la asignatura es impartida a personas de edades muy tempranas (desde 8 a 14 años generalmente), de hecho, no es importante tan solo crear un diseño llamativo sino que también la propia discriminación de colores u otras funcionalidades del tipo pueden ayudar a obtener una serie de enseñanzas concretas al alumnado, además de ayudar a personas con distintos tipos de problemas visuales u otras patologías y, sobre todo, que sea sencillo de usar para que pueda interactuar el alumnado y para que pueda ser gestionada por el profesorado sin demasiada formación en las TIC .
3. Las características señaladas, por tanto, podrían ser, el desarrollo de cuestionarios, atender al diseño responsive, usabilidad y seguimiento y evaluación del alumnado.
4. Actualmente, el principal problema de las herramientas no es la falta de funcionalidades, sino el hecho de que no pueden abarcar la asignatura por sí solas, en consecuencia, es necesario aunar las funcionalidades más básicas para la asignatura en una sola herramienta que, además, permita establecer un sistema de evaluación para que pueda ser el epicentro de la enseñanza de la asignatura que, además, sea fácil de manejar por todo tipo de usuarios.

6 DEFINICIÓN DE LA SOLUCIÓN

La investigación y el estudio bibliográfico han dejado palpables una serie de necesidades que deben ser cubiertas por la solución a desarrollar pero con un nivel de abstracción muy alto, por tanto, se debe transformar la información a un nivel más objetivo. Para ello se ha creado, en primer lugar, una especificación formal donde se han redactado todos los aspectos de la aplicación de una forma mucho más metódica y se han establecido unos objetivos funcionales resumidos, combinando así el lenguaje natural en el que se encuentra la información obtenida previamente con la objetividad a la que se desea llegar en el siguiente paso.

Posteriormente, esta información ha sido plasmada en un diagrama de casos de uso del sistema que permite entender de una forma más visual y esquemática el funcionamiento de la aplicación y se han descrito en detalle cada uno de los actores y casos de uso junto con sus requisitos funcionales asociados, incluyendo la secuencia de pasos que garantizan su correcto cumplimiento, además, también se han abordado los requisitos no funcionales que condicionan la calidad y comportamiento esperado del sistema. Por último, se han definido las tecnologías a utilizar y la arquitectura de la solución para tener una visión global de la solución a desarrollar antes de comenzar con el desarrollo de la misma.

6.1 ESPECIFICACIÓN FORMAL DE LA SOLUCIÓN

El sistema permitirá el acceso mediante un mecanismo de inicio de sesión en el que podrán autenticarse tres tipos de usuarios: administrador, profesor y alumno. Cada usuario tendrá acceso únicamente a las funcionalidades correspondientes a su rol, sin posibilidad de utilizar las funciones asignadas a otros.

No existirá un proceso de registro automático. El administrador será creado de forma manual y, una vez dentro de la aplicación, tendrá la capacidad de crear las cuentas de los profesores. Para ello, deberá introducir los siguientes datos: nombre, primer apellido, segundo apellido y dirección de correo electrónico. Se realizará una comprobación de validez de los datos introducidos (formato correcto del nombre, de los apellidos y del correo electrónico) y una verificación de unicidad del correo electrónico en la base de datos.

En el proceso de creación, el sistema generará automáticamente un nombre de usuario a partir de la primera letra del nombre y de los apellidos. Por ejemplo, para un usuario denominado José Luis Garrido Castro, el identificador generado será jgc. En caso de que existan coincidencias con identificadores ya presentes en la base de datos, se añadirá un número consecutivo equivalente al total de usuarios con la misma raíz más uno, de modo que si ya existen 40 usuarios con identificadores iniciados en jgc, el nuevo identificador será jgc41. La contraseña será generada automáticamente mediante el uso de un servicio

externo de generación de contraseñas seguras. Una vez creadas las credenciales, estas serán enviadas por correo electrónico al profesor.

El profesor, a su vez, tendrá la capacidad de crear cuentas de alumnos mediante un procedimiento equivalente al descrito para la creación de profesores.

Además, el profesor podrá crear grupos de clase, añadir alumnos a los mismos, eliminarlos o eliminar grupos completos. Un alumno podrá pertenecer a varios grupos de forma simultánea.

También podrá enviar mensajes a unos alumnos o cursos completos determinados y otra ventana donde podrá adjuntar las calificaciones generales del curso seleccionando a un alumno en concreto.

El sistema dispondrá de un área denominada administración en la que estarán disponibles las operaciones mencionadas anteriormente.

Fuera de esta sección, la aplicación contará con cuatro apartados denominados **Ritmo, Entonación, Audición y Teoría**. Cada apartado estará compuesto por dos secciones diferenciadas:

1. **Libro del curso:** permitirá al profesor subir un único archivo en formato PDF correspondiente al curso seleccionado. Si se sube un nuevo archivo, el anterior será sustituido. El profesor podrá eliminar o reemplazar el archivo en cualquier momento.
2. **Tareas a realizar:** permitirá al profesor crear tareas asociadas a un curso determinado. Cada tarea constará de una descripción y, opcionalmente, de un material de apoyo en formato PDF, MP3 o MP4. También se definirá una fecha límite de entrega. Las tareas podrán calificarse individualmente, modificarse y/o eliminarse en cualquier momento, pero no se eliminarán automáticamente al llegar la fecha de entrega; se mantendrán hasta la finalización del curso. En el apartado de Teoría, las tareas podrán sustituirse (o no) por un cuestionario tipo test. Dicho cuestionario permitirá la creación de preguntas con opciones de respuesta, y cada pregunta podrá ir acompañada de un archivo en formato MP3.

Los alumnos dispondrán de un área específica de notificaciones. En ella recibirán avisos cuando sean añadidos o eliminados de un curso o grupo, cuando se cree una nueva tarea, cuando se suba un libro del curso, cuando reciban una calificación (notificaciones del sistema) o un mensaje (notificaciones del profesor). Las notificaciones serán interactivas, permitiendo acceder directamente al recurso correspondiente.

Asimismo, los alumnos podrán acceder a los mismos cuatro apartados (Ritmo, Entonación, Audición y Teoría). En cada apartado se mostrarán las mismas dos secciones:

- En la sección Libro del curso, los alumnos podrán visualizar el archivo PDF correspondiente y descargarlo, pero no modificarlo.
- En la sección Tareas a realizar, los alumnos podrán consultar la lista de tareas disponibles y realizar las entregas correspondientes. Los formatos permitidos serán PDF, MP3 o MP4, y/o bien un comentario textual dentro de la propia aplicación. En el caso de cuestionarios, el alumno accederá

a una ventana específica donde podrá completarlo. El cuestionario no se considerará entregado hasta que el alumno pulse expresamente el botón de envío.

6.2 OBJETIVOS FUNCIONALES DEL SISTEMA

El objetivo principal del sistema es facilitar y mejorar la enseñanza del lenguaje musical mediante una plataforma que proporcione funcionalidades específicas tanto para profesores como para alumnos. Estos objetivos funcionales guían el desarrollo del sistema y aseguran que las necesidades pedagógicas y administrativas se cubran de forma eficiente y adaptada a las diferentes ramas del aprendizaje musical. Los objetivos funcionales que se persiguen son los siguientes:

- Permitir al profesorado crear, gestionar y visualizar grupos de alumnos, facilitando la organización y seguimiento académico.
- Gestionar el contenido educativo estructurado según las ramas del lenguaje musical: teoría, entonación, ritmo y audición.
- Proporcionar herramientas para que el profesor pueda crear, modificar, cerrar y eliminar tareas y cuestionarios específicos para cada rama.
- Facilitar la interacción del alumnado con las tareas y cuestionarios, incluyendo la entrega de trabajos y la realización de pruebas con autocorrección cuando corresponda.
- Permitir al profesorado revisar, calificar y gestionar las entregas y calificaciones de los alumnos de forma clara y accesible.
- Implementar un sistema de mensajería interna que facilite la comunicación entre profesores y alumnos, con notificaciones automáticas vinculadas a eventos relevantes.
- Permitir la gestión del material didáctico asociado a cada rama del aprendizaje musical, con capacidad de adaptación y personalización según las necesidades específicas de cada grupo de alumnos.

Estos objetivos funcionales proporcionan un marco claro para la posterior definición de los casos de uso y requisitos funcionales, asegurando que el sistema responda adecuadamente a las necesidades del entorno educativo para el que está diseñado.

6.3 DIAGRAMA UML DE CASOS DE USO DEL SISTEMA

Una vez definida la solución mediante lenguaje natural, se da el siguiente paso hacia el modelado de casos de uso.

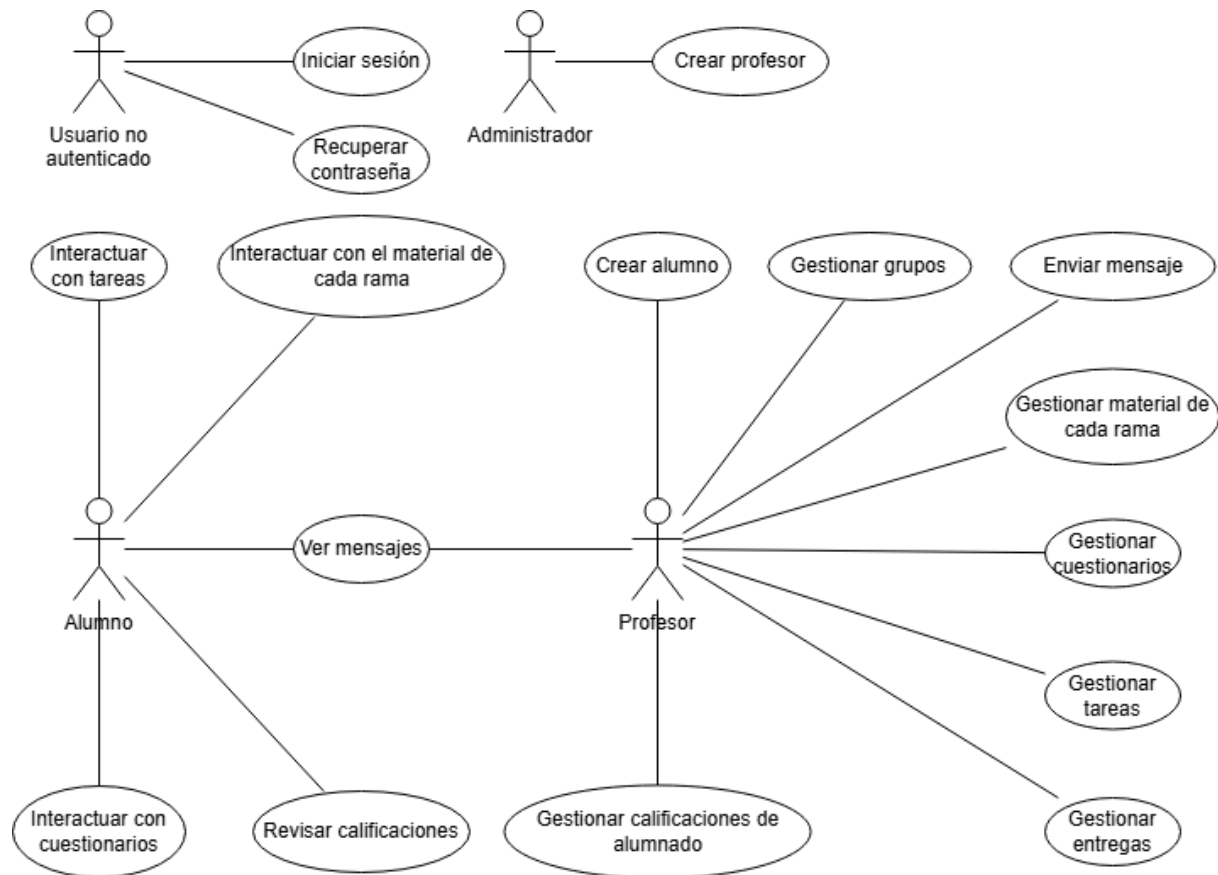


Figura 6.1: Diagrama UML de casos de uso del sistema

6.4 DEFINICIÓN DEL DIAGRAMA DE CASOS DE USO

El diagrama de casos de uso aporta una visión muy genérica y poco detallada y, aunque es mucho más objetiva que el lenguaje natural, se encuentra en un nivel de abstracción muy alto, por lo tanto, es necesario entrar en más detalle para cada uno de los elementos del diagrama.

6.4.1 Actores

Se pueden observar cuatro actores principales, cada uno con su respectivo rol:

ID del actor	ACT-1
Nombre del actor	Usuario no autenticado
Rol	Usuario que accede al sistema con el único objetivo de iniciar sesión.

Tabla 6.1: Actor 1

ID del actor	ACT-2
Nombre del actor	Administrador
Rol	Usuario autenticado con el objetivo de crear a los usuarios con rol de profesor en el sistema.

Tabla 6.2: Actor 2

ID del actor	ACT-3
Nombre del actor	Profesor
Rol	Usuario autenticado con el objetivo de crear usuarios con el rol alumno y gestionar el contenido académico dirigido a los mismos en el sistema.

Tabla 6.3: Actor 3

ID del actor	ACT-4
Nombre del actor	Alumno
Rol	Usuario autenticado con el objetivo de acceder al contenido e interactuar con el mismo a través del sistema.

Tabla 6.4: Actor 4

6.4.2 Casos de uso

Para definir los casos de uso del sistema la mejor forma de hacerlo es, como en cualquier otro proyecto de desarrollo de software, mediante los requisitos funcionales que lo conforman.

Definición general

Nombre	Iniciar sesión
Descripción	Permitir al usuario no autenticado iniciar sesión en el sistema proporcionando sus credenciales y, en caso de éxito, tendrá acceso a las funcionalidades del actor que tenga asignado en el sistema.
Actor	ACT-1
Requisitos funcionales	RF-01

Tabla 6.5: Caso de uso: Iniciar sesión

Nombre	Recuperar contraseña
Descripción	Permitir al usuario recuperar el acceso a su cuenta mediante la verificación de su dirección de correo.
Actor	ACT-1
Requisitos funcionales	RF-02

Tabla 6.6: Caso de uso: Recuperar contraseña

Nombre	Crear profesor
Descripción	Permitir al administrador crear nuevos usuarios con rol de profesor en el sistema, proporcionando los datos necesarios y validando la información.
Actor	ACT-2
Requisitos funcionales	RF-03

Tabla 6.7: Caso de uso: Crear profesor

Nombre	Crear alumno
Descripción	Permitir al profesor crear nuevos usuarios con rol de alumno en el sistema, proporcionando los datos necesarios y validando la información.
Actor	ACT-3
Requisitos funcionales	RF-05 RF-04

Tabla 6.8: Caso de uso: Crear alumno

Nombre	Gestionar grupos
Descripción	Permitir al profesor crear nuevos grupos, revisar los que ha creado y elegir un grupo para trabajar con él.
Actor	ACT-3
Requisitos funcionales	RF-06 RF-07 RF-08

Tabla 6.9: Caso de uso: Gestionar grupos

Nombre	Enviar mensaje
Descripción	Permitir al profesor enviar mensajes a los alumnos de un grupo seleccionado.
Actor	ACT-3
Requisitos funcionales	RF-09

Tabla 6.10: Caso de uso: Enviar mensaje

Nombre	Gestionar material de cada rama
Descripción	Permitir al profesor gestionar el material de cada rama de la asignatura.
Actor	ACT-3
Requisitos funcionales	RF-10 RF-11 RF-12

Tabla 6.11: Caso de uso: Gestionar material de cada rama

Nombre	Gestionar tareas
Descripción	Permitir al profesor gestionar las tareas de cada rama de la asignatura.
Actor	ACT-3
Requisitos funcionales	RF-13 RF-14 RF-15 RF-16 RF-17

Tabla 6.12: Caso de uso: Gestionar tareas

Nombre	Gestionar cuestionarios
Descripción	Permitir al profesor gestionar los cuestionarios de la rama de teoría de la asignatura.
Actor	ACT-3
Requisitos funcionales	RF-18 RF-19 RF-20 RF-21 RF-22

Tabla 6.13: Caso de uso: Gestionar entregas

Nombre	Gestionar entregas
Descripción	Permitir al profesor gestionar las entregas de las tareas de los alumnos.
Actor	ACT-3
Requisitos funcionales	RF-23 RF-24 RF-25

Tabla 6.14: Caso de uso: Gestionar entregas

Nombre	Gestionar calificaciones del alumnado
Descripción	Permitir al profesor gestionar las calificaciones generales del curso de los alumnos.
Actor	ACT-3
Requisitos funcionales	RF-26 RF-27

Tabla 6.15: Caso de uso: Gestionar calificaciones del alumnado

Nombre	Ver mensajes
Descripción	Permitir al alumno y al profesor ver los mensajes.
Actor	ACT-3 ACT-4
Requisitos funcionales	RF-28

Tabla 6.16: Caso de uso: Ver mensajes

Nombre	Interactuar con el material de cada rama
Descripción	Permitir al alumno interactuar con el material de cada rama.
Actor	ACT-4
Requisitos funcionales	RF-10

Tabla 6.17: Caso de uso: Interactuar con el material de cada rama

Nombre	Revisar calificaciones
Descripción	Permitir al alumno revisar las calificaciones.
Actor	ACT-4
Requisitos funcionales	RF-25

Tabla 6.18: Caso de uso: Revisar calificaciones

Nombre	Interactuar con tareas
Descripción	Permitir al alumno interactuar con las tareas.
Actor	ACT-4
Requisitos funcionales	RF-13 RF-29

Tabla 6.19: Caso de uso: Interactuar con tareas

Nombre	Interactuar con cuestionarios
Descripción	Permitir al alumno interactuar con los cuestionarios.
Actor	ACT-4
Requisitos funcionales	RF-18 RF-30

Tabla 6.20: Caso de uso: Interactuar con cuestionarios

Requisitos funcionales

RF-01	Iniciar sesión
Descripción	Permitir al usuario no autenticado iniciar sesión en el sistema proporcionando sus credenciales y, en caso de éxito, tendrá acceso a las funcionalidades del actor que tenga asignado en el sistema.
Precondición	El usuario no debe estar autenticado en el sistema.
Secuencia de pasos	1. El usuario accede a la pantalla de inicio de sesión. 2. El usuario introduce su nombre de usuario y contraseña. 3. El sistema valida las credenciales introducidas. 4. Si las credenciales son correctas, el sistema autentica al usuario. 5. El sistema asigna al usuario el rol correspondiente y concede acceso a las funcionalidades habilitadas. 6. El usuario es redirigido a la página principal o panel correspondiente a su rol.
Postcondición	El usuario queda autenticado en el sistema con acceso a las funcionalidades correspondientes a su rol.
Excepciones	■ El usuario introduce credenciales incorrectas. ■ Problemas de conexión con la base de datos.

Tabla 6.21: Requisito funcional: RF-01 Iniciar sesión

RF-02	Recuperar contraseña
Descripción	Permitir al usuario recuperar el acceso a su cuenta mediante la verificación de su dirección de correo.
Precondición	El usuario debe tener acceso al correo electrónico asociado a su cuenta y no estar autenticado en el sistema.
Secuencia de pasos	1. El usuario accede a la opción "Recuperar contraseña" desde la pantalla de inicio de sesión. 2. El usuario introduce su dirección de correo electrónico registrada. 3. El sistema verifica que el correo electrónico existe en la base de datos. 4. El sistema actualiza la contraseña. 5. El sistema envía la contraseña por correo y notifica al usuario del éxito.
Postcondición	El usuario puede iniciar sesión con la nueva contraseña establecida.
Excepciones	■ El correo electrónico proporcionado no está registrado en el sistema. ■ El sistema no puede conectarse a la base de datos o servicio de correo.

Tabla 6.22: Requisito funcional: RF-02 Recuperar contraseña

RF-03	Crear profesor
Descripción	Permitir al administrador crear nuevos usuarios con rol de profesor en el sistema, proporcionando los datos necesarios.
Precondición	El administrador debe estar autenticado en el sistema.
Secuencia de pasos	1. El administrador introduce los datos requeridos del profesor (nombre, apellidos y correo). 2. El sistema valida los datos mediante RF-04. 3. El sistema guarda el nuevo profesor. 4. El sistema notifica al administrador que el profesor ha sido creado correctamente.
Postcondición	El nuevo profesor queda registrado en el sistema y puede iniciar sesión con las credenciales asignadas.
Excepciones	■ Datos incompletos o inválidos introducidos por el administrador. ■ Problemas de conexión con la base de datos.

Tabla 6.23: Requisito funcional: RF-03 Crear profesor

RF-04	Validación y generación automática de credenciales
Descripción	Validar la información de creación de un nuevo usuario comprobando la unicidad del correo, formatos específicos para nombre, apellidos y correo, y generar automáticamente el username y la contraseña.
Precondición	Se reciben datos completos del profesor para validar y generar credenciales.
Secuencia de pasos	1. El sistema comprueba que el correo no está registrado. 2. El sistema valida que el correo tenga formato tipo ejemplo@ejemplo.com. 3. El sistema valida que el nombre y apellidos solo contengan letras sin espacios ni caracteres inválidos. 4. El sistema genera el username a partir de las siglas de nombre y apellidos. 5. Si el username generado ya existe, se añade un identificador numérico para diferenciarlo. 6. El sistema solicita a una API externa la generación de una contraseña segura. 7. El sistema asocia el username y contraseña generados a la cuenta del profesor.
Postcondición	Se dispone de credenciales únicas y válidas para el nuevo profesor, listas para su uso en el sistema.
Excepciones	■ El correo proporcionado ya está registrado. ■ Los datos no cumplen con el formato requerido. ■ Fallo en la comunicación con la API externa para generación de contraseña. ■ Problemas técnicos al generar o asignar las credenciales.

Tabla 6.24: Requisito funcional: RF-04 Validación y generación automática de credenciales

RF-05	Crear alumno
Descripción	Permitir al profesor crear nuevos usuarios con rol de alumno en el sistema, proporcionando los datos necesarios.
Precondición	El profesor debe estar autenticado en el sistema.
Secuencia de pasos	1. El profesor introduce los datos requeridos del alumno (nombre, apellidos y correo). 2. El sistema valida los datos mediante RF-04. 3. El sistema guarda el nuevo alumno. 4. El sistema notifica al profesor que el alumno ha sido creado correctamente.
Postcondición	El nuevo alumno queda registrado en el sistema y puede iniciar sesión con las credenciales asignadas.
Excepciones	■ Datos incompletos o inválidos introducidos por el profesor. ■ Problemas de conexión con la base de datos.

Tabla 6.25: Requisito funcional: RF-05 Crear alumno

RF-06	Crear grupo
Descripción	Permitir al profesor crear nuevos grupos proporcionando la información necesaria (nombre del grupo y alumnos).
Precondición	El profesor debe estar autenticado en el sistema y tener usuarios creados.
Secuencia de pasos	1. El profesor accede a la sección de administración. 2. El profesor introduce los datos requeridos para el grupo. 3. El sistema valida la información introducida. 4. El sistema guarda el nuevo grupo. 5. El sistema crea las ramas de la asignatura vacías (sin ningún material aún) y las guarda. 6. El sistema notifica al profesor que el grupo ha sido creado correctamente.
Postcondición	El grupo queda registrado en el sistema asociado al profesor y los alumnos incluidos.
Excepciones	■ Datos inválidos introducidos por el profesor. ■ Problemas de conexión con la base de datos.

Tabla 6.26: Requisito funcional: RF-06 Crear grupo

RF-07	Revisar grupos
Descripción	Permitir al profesor visualizar la lista de grupos que ha creado previamente.
Precondición	El profesor debe estar autenticado en el sistema y tener grupos creados.
Secuencia de pasos	1. El profesor accede a la sección de grupos. 2. El sistema recupera y muestra la lista de grupos asociados al profesor. 3. El profesor revisa los grupos mostrados.
Postcondición	El profesor puede ver y seleccionar cualquier grupo de su lista para gestionar.
Excepciones	■ No existen grupos creados por el profesor. ■ Problemas de conexión con la base de datos.

Tabla 6.27: Requisito funcional: RF-07 Listar grupos

RF-08	Seleccionar grupo
Descripción	Permitir al profesor seleccionar un grupo creado para realizar acciones específicas o trabajar con su contenido.
Precondición	El profesor debe estar autenticado y haber creado al menos un grupo.
Secuencia de pasos	1. El profesor visualiza la lista de grupos creados mediante RF-07. 2. El profesor selecciona un grupo de la lista. 3. El sistema carga los datos y funcionalidades correspondientes al grupo seleccionado. 4. El profesor puede comenzar a trabajar con el grupo.
Postcondición	El sistema queda configurado para trabajar con el grupo seleccionado.
Excepciones	■ El grupo seleccionado no existe. ■ Problemas de carga o acceso a los datos del grupo.

Tabla 6.28: Requisito funcional: RF-08 Seleccionar grupo

RF-09	Enviar mensaje
Descripción	Permitir al profesor enviar mensajes a los alumnos de un grupo seleccionado.
Precondición	El profesor debe estar autenticado y tener un grupo seleccionado.
Secuencia de pasos	1. El profesor accede a administración o a perfil y entra en la herramienta de creación de mensajes. 2. El profesor redacta el mensaje, el asunto y selecciona los alumnos del grupo seleccionado como destinatarios. 3. El profesor envía el mensaje. 4. El sistema valida los datos. 5. El sistema guarda el mensaje y confirma el envío exitoso del mensaje al profesor.
Postcondición	Los alumnos del grupo y el profesor reciben el mensaje en su bandeja de entrada dentro de la plataforma.
Excepciones	■ No se han seleccionado alumnos y/o el mensaje y/o el asunto no son válidos. ■ Problemas de conexión a la base de datos.

Tabla 6.29: Requisito funcional: RF-09 Enviar mensaje

RF-10	Ver libro del curso
Descripción	Permitir al profesor y al alumno visualizar el libro asociado a una rama específica de la asignatura.
Precondición	El profesor o el alumno deben estar autenticados, tener un grupo elegido y existir un libro asociado a la rama seleccionada.
Secuencia de pasos	1. El usuario selecciona la rama de la asignatura. 2. El sistema muestra el libro actual asociado a dicha rama. 3. El usuario revisa el contenido del libro.
Postcondición	El usuario visualiza correctamente el libro asociado a la rama seleccionada pudiendo revisarlo desde el propio sistema o descargarlo.
Excepciones	■ No existe libro asociado a la rama seleccionada. ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 6.30: Requisito funcional: RF-10 Ver libro del curso

RF-11	Subir o reemplazar libro del curso
Descripción	Permitir al profesor subir un nuevo libro o reemplazar el libro existente asociado a una rama de la asignatura.
Precondición	El profesor debe estar autenticado y tener un grupo seleccionado.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El sistema muestra el libro actual asociado, si existe. 3. El profesor selecciona un archivo local para subir o reemplazar el libro. 4. El sistema valida el formato y tamaño del archivo. 5. El sistema actualiza el libro asociado a la rama con el nuevo archivo. 6. El sistema informa de la actualización exitosa.
Postcondición	El libro asociado a la rama queda actualizado con el nuevo archivo.
Excepciones	■ Archivo no válido (formato o tamaño). ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 6.31: Requisito funcional: RF-11 Subir o reemplazar libro del curso

RF-12	Eliminar libro del curso
Descripción	Permitir al profesor eliminar el libro asociado a una rama de la asignatura.
Precondición	El profesor debe estar autenticado, tener un grupo seleccionado y existir un libro asociado a la rama seleccionada.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El sistema muestra el libro actual asociado a la rama. 3. El profesor solicita eliminar el libro. 4. El sistema solicita confirmación de la acción. 5. El profesor confirma la eliminación. 6. El sistema elimina el libro asociado a la rama. 7. El sistema informa de la eliminación exitosa.
Postcondición	El libro asociado a la rama es eliminado y ya no está disponible para consulta.
Excepciones	■ No se ha elegido ningún grupo. ■ No existe libro para eliminar en la rama seleccionada. ■ Problemas de conexión a la base de datos.

Tabla 6.32: Requisito funcional: RF-12 Eliminar libro del curso

RF-13	Revisar tareas
Descripción	Permitir al profesor y al alumno revisar la lista de tareas creadas para una rama específica de la asignatura.
Precondición	El profesor o el alumno deben estar autenticados, tener un grupo seleccionado y existir al menos una tarea creada en la rama seleccionada.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El sistema muestra la lista de tareas disponibles. 3. El profesor puede seleccionar una tarea para ver sus detalles.
Postcondición	El profesor visualiza correctamente la información de las tareas existentes.
Excepciones	■ No se ha elegido ningún grupo. ■ No existen tareas en la rama seleccionada. ■ Problemas de conexión a la base de datos.

Tabla 6.33: Requisito funcional: RF-13 Revisar tareas

RF-14	Crear tarea
Descripción	Permitir al profesor crear nuevas tareas para una rama específica de la asignatura.
Precondición	El profesor debe estar autenticado y tener un grupo seleccionado.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El profesor accede al formulario para crear una tarea. 3. El profesor introduce los detalles de la tarea (título, descripción, fecha límite, alumnos dentro del grupo a los que se dirige la tarea y un archivo de apoyo opcional). 4. El profesor confirma la creación de la tarea. 5. El sistema valida los datos introducidos, el archivo de apoyo si lo hubiera (formato correcto) y que haya alumnos seleccionados. 6. El sistema guarda la tarea, envía un mensaje del sistema a los alumnos e informa al profesor de la creación exitosa.
Postcondición	La nueva tarea queda registrada y disponible en la lista de tareas de la rama y el alumno queda avisado de ello en los mensajes (RFT-1).
Excepciones	■ No se ha elegido ningún grupo. ■ Datos incompletos o inválidos en el formulario. ■ Problemas de conexión a la base de datos.

Tabla 6.34: Requisito funcional: RF-14 Crear tarea

RF-15	Modificar tarea
Descripción	Permitir al profesor modificar las tareas existentes en una rama específica de la asignatura.
Precondición	El profesor debe estar autenticado, tener un grupo seleccionado y existir la tarea que desea modificar.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El profesor selecciona la tarea a modificar. 3. El sistema muestra los datos actuales de la tarea. 4. El profesor edita los detalles necesarios. 5. El profesor confirma la modificación. 6. El sistema valida los datos introducidos, el archivo de apoyo si lo hubiera (formato correcto) y que haya alumnos seleccionados. 7. El sistema actualiza la tarea, envía un mensaje del sistema a los alumnos e informa al profesor de la modificación exitosa.
Postcondición	La tarea queda actualizada con los nuevos datos proporcionados y el alumno queda avisado de ello en los mensajes (RFT-1).
Excepciones	■ No se ha elegido ningún grupo. ■ Datos incompletos o inválidos en el formulario. ■ Problemas de conexión a la base de datos.

Tabla 6.35: Requisito funcional: RF-15 Modificar tarea

RF-16	Cerrar tarea
Descripción	Permitir al profesor cerrar una tarea, impidiendo que los alumnos sigan entregándola.
Precondición	El profesor debe estar autenticado, tener un grupo seleccionado y la tarea debe existir y estar activa (fecha de cierre no cumplida o ya cerrada antes).
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El profesor selecciona la tarea que desea cerrar. 3. El sistema muestra la información de la tarea. 4. El profesor solicita cerrar la tarea. 5. El sistema cierra la tarea e informa de la acción exitosa.
Postcondición	La tarea queda cerrada y no acepta nuevas entregas de los alumnos.
Excepciones	■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 6.36: Requisito funcional: RF-16 Cerrar tarea

RF-17	Eliminar tarea
Descripción	Permitir al profesor eliminar tareas existentes de una rama de la asignatura.
Precondición	El profesor debe estar autenticado tener un grupo seleccionado y la tarea debe existir.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El profesor selecciona la tarea que desea eliminar. 3. El sistema solicita confirmación para eliminar la tarea. 4. El profesor confirma la eliminación. 5. El sistema elimina la tarea e informa de la acción exitosa.
Postcondición	La tarea queda eliminada y ya no está disponible para los usuarios.
Excepciones	■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 6.37: Requisito funcional: RF-17 Eliminar tarea

RF-18	Revisar cuestionarios
Descripción	Permitir al profesor y al alumno revisar la lista de cuestionarios de la rama de teoría de la asignatura.
Precondición	El profesor o el alumno deben de estar autenticados, tener un grupo seleccionado y existir al menos un cuestionario creado en la rama de teoría.
Secuencia de pasos	1. El usuario selecciona la rama de teoría de la asignatura. 2. El usuario accede a la sección de cuestionarios. 3. El sistema muestra la lista de cuestionarios disponibles. 4. El profesor puede seleccionar un cuestionario para ver sus detalles.
Postcondición	El profesor visualiza correctamente la información de los cuestionarios existentes.
Excepciones	■ No existen cuestionarios en la rama seleccionada. ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 6.38: Requisito funcional: RF-18 Revisar cuestionarios

RF-19	Crear cuestionario
Descripción	Permitir al profesor crear nuevos cuestionarios para la rama de teoría de la asignatura.
Precondición	El profesor debe estar autenticado y tener un grupo seleccionado.
Secuencia de pasos	1. El profesor selecciona la rama de teoría de la asignatura. 2. El usuario accede a la sección de cuestionarios. 3. El profesor accede al formulario para crear un cuestionario. 4. El profesor introduce los detalles del cuestionario (título, alumnos, fecha de cierre y preguntas tipo test, a su vez, cada pregunta podrá tener un material de apoyo opcional). 5. El profesor confirma la creación del cuestionario. 6. El sistema valida los datos introducidos, las preguntas y sus correspondientes materiales de apoyo (formato), si lo hubiese. 7. El sistema guarda el cuestionario, envía un mensaje del sistema a los alumnos e informa al profesor de la creación exitosa.
Postcondición	El nuevo cuestionario queda registrado y disponible en la lista de cuestionarios de la rama y el alumno queda avisado de ello en los mensajes (RFT-1).
Excepciones	■ Datos incompletos o inválidos en el formulario. ■ Problemas de conexión a la base de datos.

Tabla 6.39: Requisito funcional: RF-19 Crear cuestionario

RF-20	Modificar cuestionario
Descripción	Permitir al profesor modificar los cuestionarios existentes en la rama de teoría de la asignatura.
Precondición	El profesor debe estar autenticado, debe de haber un grupo seleccionado y existir el cuestionario que se desea modificar.
Secuencia de pasos	1. El profesor selecciona la rama de teoría de la asignatura. 2. El usuario accede a la sección de cuestionarios. 3. El profesor selecciona el cuestionario a modificar. 4. El sistema muestra los datos actuales del cuestionario. 5. El profesor edita los detalles necesarios. 6. El profesor confirma la modificación. 7. El sistema valida los datos introducidos, las preguntas y sus correspondientes materiales de apoyo (formato), si lo hubiese. 8. El sistema actualiza el cuestionario, envía un mensaje del sistema a los alumnos e informa al profesor de la modificación exitosa.
Postcondición	El cuestionario queda actualizado con los nuevos datos proporcionados y el alumno queda avisado de ello en los mensajes (RFT-1).
Excepciones	■ No se ha elegido ningún grupo. ■ Datos inválidos o incompletos durante la modificación. ■ Problemas de conexión a la base de datos.

Tabla 6.40: Requisito funcional: RF-20 Modificar cuestionario

RF-21	Cerrar cuestionario
Descripción	Permitir al profesor cerrar un cuestionario, impidiendo que los alumnos sigan respondiéndolo.
Precondición	El profesor debe estar autenticado, debe de haber un grupo seleccionado y el cuestionario debe existir y estar activo.
Secuencia de pasos	1. El profesor selecciona la rama de teoría de la asignatura. 2. El profesor selecciona el cuestionario que desea cerrar. 3. El sistema solicita confirmación para cerrar el cuestionario. 4. El profesor confirma el cierre. 5. El sistema cierra el cuestionario e informa de la acción exitosa.
Postcondición	El cuestionario queda cerrado y no acepta nuevas respuestas de los alumnos.
Excepciones	■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 6.41: Requisito funcional: RF-21 Cerrar cuestionario

RF-22	Eliminar cuestionario
Descripción	Permitir al profesor eliminar cuestionarios existentes en la rama de teoría de la asignatura.
Precondición	El profesor debe estar autenticado, debe de haber un grupo seleccionado y el cuestionario debe existir.
Secuencia de pasos	1. El profesor selecciona la rama de teoría de la asignatura. 2. El profesor selecciona el cuestionario que desea eliminar. 3. El sistema solicita confirmación para eliminar el cuestionario. 4. El profesor confirma la eliminación. 5. El sistema elimina el cuestionario y confirma la acción exitosa.
Postcondición	El cuestionario queda eliminado y ya no está disponible para los usuarios.
Excepciones	■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 6.42: Requisito funcional: RF-22 Eliminar cuestionario

RF-23	Revisar entregas
Descripción	Permitir al profesor revisar las entregas realizadas por los alumnos para una tarea específica.
Precondición	El profesor debe estar autenticado, tener un grupo seleccionado, existir una tarea creada y haber al menos una entrega realizada por los alumnos.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El profesor selecciona una tarea existente. 3. El sistema muestra la lista de entregas realizadas por los alumnos. 4. El profesor selecciona una entrega para visualizar su contenido.
Postcondición	El profesor visualiza correctamente las entregas asociadas a la tarea seleccionada.
Excepciones	■ No existen entregas para la tarea seleccionada. ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 6.43: Requisito funcional: RF-23 Revisar entregas

RF-24	Calificar entrega
Descripción	Permitir al profesor calificar una entrega realizada por un alumno para una tarea específica.
Precondición	El profesor debe estar autenticado, debe de haber un grupo seleccionado y existir una entrega asociada a la tarea seleccionada.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El profesor selecciona una tarea. 3. El profesor selecciona una entrega concreta. 4. El sistema muestra el contenido de la entrega. 5. El profesor introduce la calificación. 6. El profesor confirma la calificación. 7. El sistema valida la calificación introducida (Número real del 1 al 10). 8. El sistema guarda la calificación asignada, envía un mensaje del sistema a los alumnos e informa al profesor del éxito en calificar.
Postcondición	La entrega queda calificada, la calificación se almacena en el sistema y el alumno queda avisado de ello en los mensajes (RFT-1).
Excepciones	■ No existen entregas para la tarea seleccionada. ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 6.44: Requisito funcional: RF-24 Calificar entrega

RF-25	Revisar calificaciones de entregas
Descripción	Permitir al profesor revisar las calificaciones asignadas a las entregas de una tarea específica.
Precondición	El profesor debe estar autenticado, debe de haber un grupo seleccionado y existir una entrega asociada a la tarea seleccionada.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El profesor selecciona una tarea. 3. El sistema muestra la lista de entregas calificadas junto con sus calificaciones.
Postcondición	El profesor visualiza correctamente las calificaciones asociadas a las entregas.
Excepciones	■ No existen entregas calificadas para la tarea seleccionada. ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 6.45: Requisito funcional: RF-25 Revisar calificaciones de entregas

RF-26	Revisar calificaciones generales del alumno
Descripción	Permitir al profesor o al alumno revisar las calificaciones generales obtenidas previamente por un alumno en el curso.
Precondición	El profesor o el alumno deben de estar autenticados, tener un grupo seleccionado y que dicho alumno tenga calificaciones generales registradas en el sistema.
Secuencia de pasos	1. El usuario accede a perfil o a administración (esta última solo si es profesor). 2. Si el usuario es alumno, el sistema identifica automáticamente al alumno autenticado. 3. Si el usuario es profesor, el sistema requiere de la selección del alumno desde la herramienta de calificaciones generales. 4. El sistema muestra las calificaciones generales registradas para el alumno. 5. El usuario revisa el historial de calificaciones.
Postcondición	El usuario visualiza correctamente las calificaciones generales del alumno.
Excepciones	■ El alumno no tiene calificaciones registradas. ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 6.46: Requisito funcional: RF-26 Revisar calificaciones generales del alumno

RF-27	Asignar calificación general al alumno
Descripción	Permitir al profesor asignar o actualizar la calificación general de un alumno en el curso.
Precondición	El profesor debe estar autenticado y tener un grupo seleccionado.
Secuencia de pasos	1. El profesor selecciona el perfil o administración. 2. El profesor accede a la herramienta de calificaciones generales. 3. El profesor selecciona un alumno del listado. 4. El sistema muestra las calificaciones generales actuales, si existen. 5. El profesor introduce o modifica la calificación general (que podrá ser Q1, Q2, Q3, Ordinaria o extraordinaria, esta última, solo se activará en caso de tener una calificación inferior a 5 en la ordinaria). 6. El profesor confirma la asignación de la calificación. 7. El sistema valida la calificación introducida (Número entero del 1 al 10). 8. El sistema guarda la calificación general del alumno, envía un mensaje del sistema a los alumnos e informa al profesor de la operación exitosa.
Postcondición	La calificación general del alumno queda registrada o actualizada en el sistema y el alumno queda avisado de ello en los mensajes (RFT-1).
Excepciones	■ La calificación introducida no es válida. ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 6.47: Requisito funcional: RF-27 Asignar calificación general al alumno



RF-28	Ver mensajes
Descripción	Permitir al alumno y al profesor visualizar los mensajes recibidos en la plataforma y marcarlos como leídos si así lo desea el usuario.
Precondición	El profesor o alumno deben de estar autenticado y tener al menos un mensaje recibido.
Secuencia de pasos	1. El usuario accede a la sección de mensajes. 2. El sistema muestra la lista de mensajes recibidos. 3. De forma opcional y si el mensaje no estaba ya leído, el usuario marca el mensaje como leído. 4. El sistema actualiza el estado del mensaje.
Postcondición	El usuario puede ver sus mensajes y marcados como leídos en caso de que el usuario lo haya indicado.
Excepciones	■ No existen mensajes para el usuario. ■ Problemas de conexión a la base de datos.

Tabla 6.48: Requisito funcional: RF-28 Ver mensajes

RF-29	
Entregar tarea	
Descripción	Permitir al alumno realizar la entrega de una tarea asignada por el profesor.
Precondición	El alumno debe estar autenticado, tener un grupo seleccionado, existir una tarea activa y encontrarse dentro del plazo de entrega.
Secuencia de pasos	1. El alumno selecciona la rama de la asignatura. 2. El alumno selecciona una tarea disponible. 3. El sistema muestra los detalles de la tarea. 4. El alumno adjunta el comentario y/o archivo requerido. 5. El alumno confirma la entrega de la tarea. 6. El sistema valida que el comentario sea válido y que el archivo adjunto (si lo hubiera) sea válido. 7. El sistema registra la entrega realizada, envía un mensaje del sistema al profesor e informa al alumno de que la entrega fue exitosa.
Postcondición	La tarea queda entregada y disponible para su posterior revisión y calificación por parte del profesor y el alumno profesor avisado de ello en los mensajes (RFT-1).
Excepciones	■ No se ha elegido ningún grupo. ■ El archivo o el comentario adjunto/s no es válido. ■ Problemas de conexión a la base de datos.

Tabla 6.49: Requisito funcional: RF-29 Entregar tarea

RF-30	Rellenar y entregar cuestionario
Descripción	Permitir al alumno rellenar un cuestionario y entregarlo para su autocorrección inmediata por el sistema.
Precondición	El alumno debe estar autenticado, tener un grupo seleccionado, existir un cuestionario activo y encontrarse dentro del plazo de entrega.
Secuencia de pasos	1. El alumno selecciona la rama de teoría de la asignatura. 2. El alumno selecciona un cuestionario disponible. 3. El sistema muestra las preguntas del cuestionario. 4. El alumno responde a las preguntas. 5. El alumno confirma la entrega del cuestionario. 6. El sistema valida que todas las preguntas han sido respondidas. 7. El sistema realiza la autocorrección del cuestionario. 8. El sistema registra la entrega y la calificación obtenida.
Postcondición	El cuestionario queda entregado y autocorregido, quedando registrada la calificación obtenida por el alumno.
Excepciones	■ No se ha elegido ningún grupo. ■ No se han respondido todas las preguntas y/o las respuestas no son válidas. ■ Problemas de conexión a la base de datos.

Tabla 6.50: Requisito funcional: RF-30 Rellenar y entregar cuestionario

Por último, es importante añadir un requisito transversal que se dispare en varios requisitos funcionales del sistema.

RFT-1	Envío de notificaciones del sistema
Descripción	Permitir al sistema enviar notificaciones automáticas a los usuarios cuando se produzcan determinadas acciones relevantes dentro de la plataforma.
Precondición	Debe producirse una acción del sistema que tenga asociada una notificación automática.
Secuencia de pasos	1. El sistema detecta la ejecución de una acción relevante. 2. El sistema identifica a los usuarios destinatarios de la notificación. 3. El sistema genera el contenido de la notificación. 4. El sistema envía la notificación al usuario correspondiente.
Postcondición	El usuario destinatario recibe una notificación asociada a la acción realizada.
Excepciones	■ No se puede identificar al usuario destinatario. ■ La acción anterior falla. ■ Problemas de conexión a la base de datos.

Tabla 6.51: Requisito funcional transversal: RFT-1 Envío de notificaciones del sistema

6.5 REQUISITOS NO FUNCIONALES

Tras definir todas las funcionalidades del sistema, así como los requisitos que las conforman (primero, saber qué debe hacer el sistema), las investigaciones y análisis previos han llevado a definir los siguientes requisitos no funcionales que debe cumplir la solución (después, saber cómo lo debe hacer):

RNF-01	Escalabilidad y mantenibilidad
Descripción	El sistema deberá diseñarse de forma que permita su evolución y mantenimiento a lo largo del tiempo, facilitando la incorporación de nuevas funcionalidades, la modificación de las existentes y la corrección de errores sin afectar negativamente al conjunto de la aplicación.
Justificación	Proyectos de esta índole siempre van a dejar aspectos pendientes de considerar, por lo tanto, tanto en su desarrollo como en posibles fases posteriores deberá poder ser aprovechada para proyectos mayores.
Criterio de cumplimiento	El sistema presenta una arquitectura modular, con separación clara de responsabilidades y una organización del código que facilita su extensión y mantenimiento.

Tabla 6.52: Requisito no funcional: RNF-01 Escalabilidad y mantenibilidad

RNF-02	Usabilidad y accesibilidad
Descripción	El sistema deberá ser usable y accesible, permitiendo a profesores y alumnos interactuar con la aplicación de forma intuitiva, independientemente de su nivel de competencia digital, y teniendo en cuenta criterios básicos de accesibilidad.
Justificación	Los usuarios finales no tienen por qué poseer un alto nivel de conocimientos informáticos y pueden existir alumnos con dificultades cognitivas o visuales, como el daltonismo, por lo que es necesario facilitar el acceso y comprensión de la interfaz.
Criterio de cumplimiento	La interfaz utiliza elementos visuales claros, navegación sencilla, contrastes adecuados y evita sobrecargar las pantallas con información innecesaria.

Tabla 6.53: Requisito no funcional: RNF-02 Usabilidad y accesibilidad

RNF-03	Diseño visual atractivo y motivador
Descripción	El sistema deberá contar con un diseño visual atractivo, coherente y consistente, que favorezca la motivación del alumnado y mejore la experiencia de usuario durante el proceso de aprendizaje.
Justificación	El estudio y las investigaciones previas realizadas indican que un diseño visual cuidado y llamativo incrementa la motivación del alumnado, especialmente en entornos educativos digitales.
Criterio de cumplimiento	La aplicación mantiene una estética uniforme, uso adecuado del color, iconografía clara y una disposición visual que facilita la comprensión del contenido.

Tabla 6.54: Requisito no funcional: RNF-03 Diseño visual atractivo y motivador

RNF-04	Portabilidad
Descripción	El sistema deberá ser portable, permitiendo su uso en distintos dispositivos y entornos, especialmente en navegadores web modernos y dispositivos móviles.
Justificación	La portabilidad favorece la escalabilidad del sistema y permite su uso en distintos contextos educativos.
Criterio de cumplimiento	La aplicación se desarrolla utilizando tecnologías multiplataforma y es accesible desde distintos dispositivos sin necesidad de configuraciones específicas.

Tabla 6.55: Requisito no funcional: RNF-04 Portabilidad

RNF-05	Seguridad
Descripción	El sistema deberá garantizar la seguridad de los datos personales y académicos de los usuarios, controlando el acceso a la información y protegiendo la integridad de los datos almacenados.
Justificación	La aplicación está orientada a entornos educativos públicos y privados, como conservatorios y escuelas de música, que requieren un tratamiento seguro de la información del alumnado y profesorado.
Criterio de cumplimiento	El sistema implementa mecanismos de autenticación, control de roles y restricciones de acceso a las funcionalidades y datos según el tipo de usuario.

Tabla 6.56: Requisito no funcional: RNF-05 Seguridad

RNF-06	Consistencia pedagógica
Descripción	El sistema deberá mantener una coherencia pedagógica alineada con las necesidades de aprendizaje del alumnado de lenguaje musical, reflejada en la organización de contenidos, navegación y funcionalidades.
Justificación	La aplicación se estructura en ramas de aprendizaje que corresponden a las distintas áreas del lenguaje musical, separando claramente las funcionalidades educativas de las de gestión, lo que facilita un flujo de aprendizaje claro y comprensible. Asimismo, determinadas funcionalidades se limitan a aquellas ramas en las que tienen sentido desde el punto de vista académico. En concreto, la creación de cuestionarios se restringe a la rama de teoría, ya que en las ramas de entonación, ritmo y audición este tipo de evaluación no resulta adecuada pedagógicamente.
Criterio de cumplimiento	La navegación y estructura del sistema siguen una lógica pedagógica uniforme, con contenidos, tareas, cuestionarios y calificaciones organizados por ramas y grupos. Las funcionalidades disponibles en cada rama se corresponden con métodos de evaluación y aprendizaje adecuados a su naturaleza académica, evitando ofrecer opciones que no tengan sentido pedagógico.

Tabla 6.57: Requisito no funcional: RNF-06 Consistencia pedagógica

RNF-07	Adaptabilidad del contenido
Descripción	El sistema deberá permitir la adaptación del contenido educativo en función de las características de cada grupo, posibilitando que el profesor configure materiales, tareas y cuestionarios específicos según las necesidades del alumnado.
Justificación	Cada grupo puede presentar ritmos de aprendizaje y necesidades distintas, por lo que es necesario ofrecer flexibilidad al profesorado para ajustar el contenido educativo sin afectar al resto de grupos.
Criterio de cumplimiento	El sistema permite asociar contenidos y actividades de forma independiente a cada grupo, manteniendo una personalización coherente del aprendizaje.

Tabla 6.58: Requisito no funcional: RNF-07 Adaptabilidad del contenido

6.6 TECNOLOGÍAS Y ARQUITECTURA DE LA SOLUCIÓN

Tras la definición detallada de los requisitos funcionales y no funcionales, se ha decidido adoptar una arquitectura basada en un modelo cliente-servidor con una separación clara entre frontend y backend, apoyada en servicios REST. Esta decisión responde tanto a criterios técnicos como a condicionantes generales del proyecto, tales como la viabilidad en un contexto académico, el uso de tecnologías que sean gratuitas o al menos tengan planes para no invertir dinero en el desarrollo, y la necesidad de facilitar la mantenibilidad, escalabilidad y evolución futura del sistema.

La arquitectura modular adoptada permite integrar servicios externos, desplegar la solución en entornos *cloud* y automatizar su ciclo de vida, garantizando una experiencia de usuario accesible y consistente, así como una gestión eficiente de los recursos y los datos. Estas decisiones se alinean con los objetivos del proyecto y con los requisitos no funcionales previamente definidos, especialmente en lo relativo a mantenibilidad, seguridad, portabilidad y adaptabilidad.

A continuación, se detallan las tecnologías seleccionadas para cada capa del sistema, junto con su justificación técnica y su relación con los objetivos, restricciones y requisitos establecidos.

6.6.1 Frontend

La interfaz de usuario se desarrolla como una aplicación web con el objetivo de maximizar la accesibilidad desde distintos dispositivos y plataformas, en consonancia con los requisitos de usabilidad y portabilidad. Asimismo, se adopta un enfoque híbrido que permite, como línea de trabajo futura, exportar la solución a aplicaciones móviles nativas sin necesidad de rediseñar el sistema.

Para ello, se ha seleccionado el framework **Ionic** [8] junto con **Angular** [9]. Ionic facilita el desarrollo multiplataforma y la reutilización de código, mientras que Angular aporta una arquitectura robusta y modular. En particular, Angular sigue el patrón **Modelo-Vista-Controlador (MVC)** [10], lo que favorece la separación de responsabilidades, la mantenibilidad del código y la escalabilidad del sistema, aspectos directamente relacionados con los requisitos no funcionales definidos.

Además, este enfoque arquitectónico facilita la aplicación de buenas prácticas de desarrollo y la incorporación de estándares de accesibilidad (como WCAG), al permitir una estructuración clara de las vistas, la lógica de negocio y la gestión de estados.

Para garantizar la calidad del frontend, se emplean herramientas de testing como **Jasmine** [11] y **Karma** [12], que permiten realizar pruebas unitarias y de integración, contribuyendo a la fiabilidad del sistema ante futuras modificaciones.



Figura 6.2: Tecnologías frontend: Ionic, Angular, Jasmine y Karma

La elección de estas tecnologías se justifica por su madurez, amplia documentación y comunidad activa, lo que refuerza la viabilidad técnica del proyecto dentro de un entorno académico y facilita su evolución futura.

6.6.2 Backend

El backend se implementa siguiendo una arquitectura RESTful, lo que permite una comunicación desacoplada con el frontend y facilita la integración con servicios externos. Esta decisión responde tanto a criterios técnicos como a la necesidad de mantener una arquitectura clara, extensible y fácilmente mantenible.

La tecnología base escogida es **Node.js** [13] junto con **Express.js** [14]. Node.js proporciona un entorno eficiente basado en JavaScript, coherente con el stack del frontend, mientras que Express.js ofrece la flexibilidad necesaria para construir APIs ligeras y escalables.

En materia de seguridad, se integra **JSON Web Tokens (JWT)** [15] para la autenticación y autorización de usuarios. Este mecanismo garantiza la identidad de los usuarios mediante tokens firmados, evitando accesos no autorizados y alineándose con buenas prácticas de seguridad ampliamente aceptadas.

Para mejorar la mantenibilidad y facilitar la comprensión del sistema, se utiliza **Swagger** [16] como herramienta de documentación de la API. Swagger permite generar documentación interactiva de forma automática, lo que resulta especialmente útil en contextos académicos y en futuras ampliaciones del sistema.

La calidad del backend se refuerza mediante el uso de herramientas de testing como **Jest** [17] y **Supertest** [18]. Jest permite realizar pruebas unitarias de la lógica de negocio, mientras que Supertest facilita la validación de los endpoints HTTP, contribuyendo a la fiabilidad y robustez del sistema.



Figura 6.3: Tecnologías backend: Node.js, Express.js, JWT, Swagger, Jest y Supertest

6.6.3 Base de datos

Se opta por una base de datos documental, concretamente **MongoDB** [19], debido a su flexibilidad en la gestión de datos y su capacidad de escalar horizontalmente. Esta elección permite adaptarse a posibles cambios en el modelo de datos sin necesidad de rediseños complejos.

MongoDB se integra de forma nativa con Node.js, facilitando el desarrollo y la gestión de la información relativa a usuarios, grupos, tareas y demás entidades del sistema.



Figura 6.4: MongoDB

Para su despliegue, se ha optado por **MongoDB Atlas**, un servicio gestionado en la nube que ofrece una solución escalable y segura, alineada con los requisitos no funcionales de seguridad y portabilidad.

6.6.4 Servicios externos

Con el objetivo de optimizar el desarrollo y mejorar la seguridad del sistema, se integran servicios externos que permiten reutilizar soluciones especializadas:

- **Mailjet** [20] para la gestión del envío de correos electrónicos y notificaciones automáticas.
- **API de generación de contraseñas** [21] para asegurar la creación de credenciales robustas sin desarrollar esta funcionalidad desde cero.



Figura 6.5: Servicios externos



6.6.5 Integración y despliegue continuo

El ciclo de vida del desarrollo se apoya en un flujo de integración y despliegue continuo mediante **GitHub Actions** [22], que automatiza la ejecución de pruebas y el despliegue del sistema tras cada cambio relevante en el repositorio.

El despliegue para el backend se realiza en la plataforma **Render** [23] integrada directamente con GitHub y para el frontend con **GitHub Pages**, lo que permite una actualización automática del entorno de producción. Esta decisión responde a criterios de viabilidad y gratuidad, adecuados al contexto académico del proyecto.



Figura 6.6: Tecnologías de integración y despliegue continuo

6.6.6 Arquitectura global de la aplicación

La arquitectura global sigue un enfoque modular y desacoplado, facilitando la escalabilidad, mantenibilidad y adaptación futura del sistema.

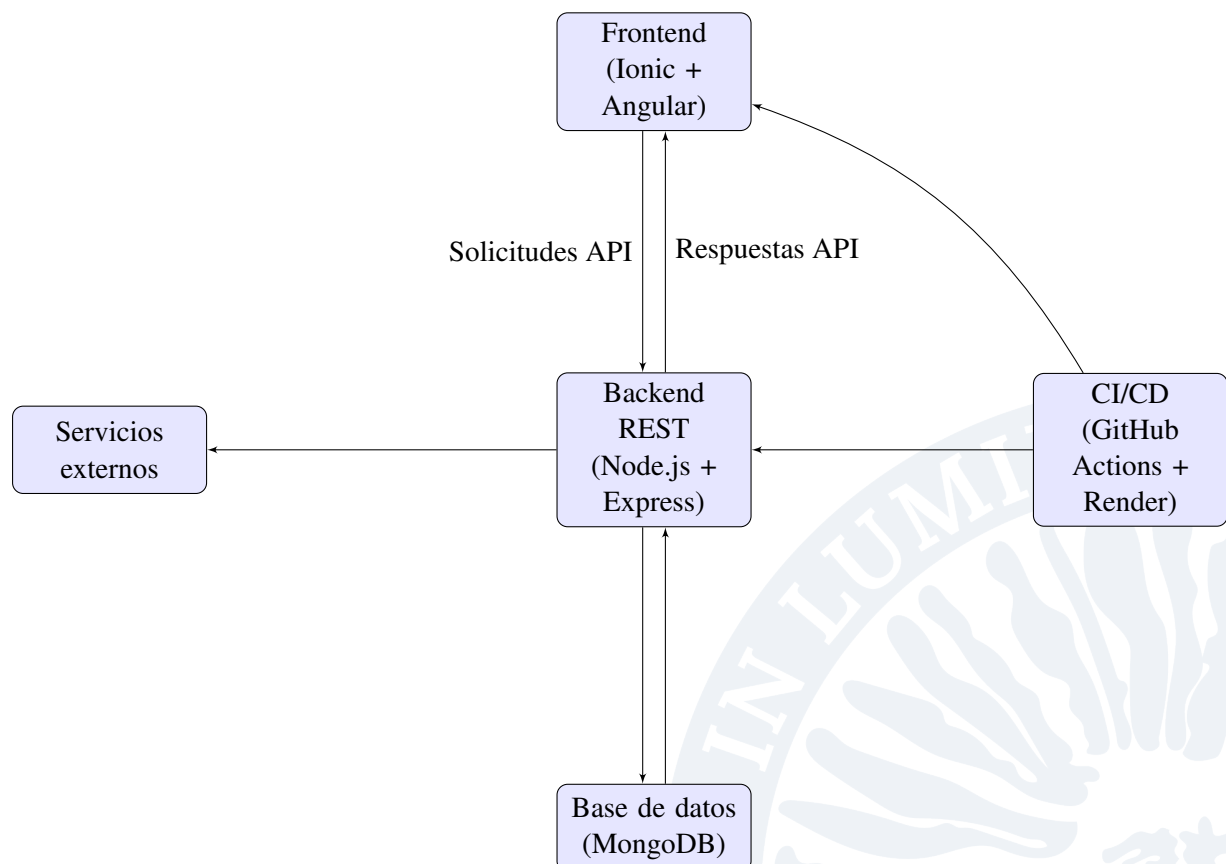


Figura 6.7: Arquitectura global de la aplicación



Con esta arquitectura y selección tecnológica, el sistema satisface los requisitos funcionales y no funcionales definidos, garantizando su viabilidad técnica en un contexto académico y dejando abiertas futuras líneas de evolución hacia entornos de producción más exigentes.

7 PLANIFICACIÓN

La planificación del proyecto se ha realizado con el objetivo de establecer una hoja de ruta clara y realista que permita organizar las tareas de forma secuencial y coordinada, facilitando así un seguimiento adecuado del progreso y la gestión del tiempo.

Se ha optado por un enfoque pragmático que prioriza la flexibilidad y adaptación a las circunstancias, sin ceñirse a una metodología de gestión específica. Esto permite ajustar las tareas y los plazos según las necesidades reales que vayan surgiendo durante el desarrollo, manteniendo un equilibrio entre el avance técnico y la calidad del resultado.

El cronograma elaborado refleja las fases principales del proyecto, dividiéndose en tareas y sub-tareas de manera que cada etapa se pueda abordar con foco y control, garantizando la coherencia y la continuidad del trabajo.

En resumen, la planificación establece un marco de referencia que orienta la ejecución del proyecto, pero deja espacio para la flexibilidad necesaria en un contexto académico y de investigación, donde el aprendizaje y la evolución del trabajo son parte fundamental del proceso.

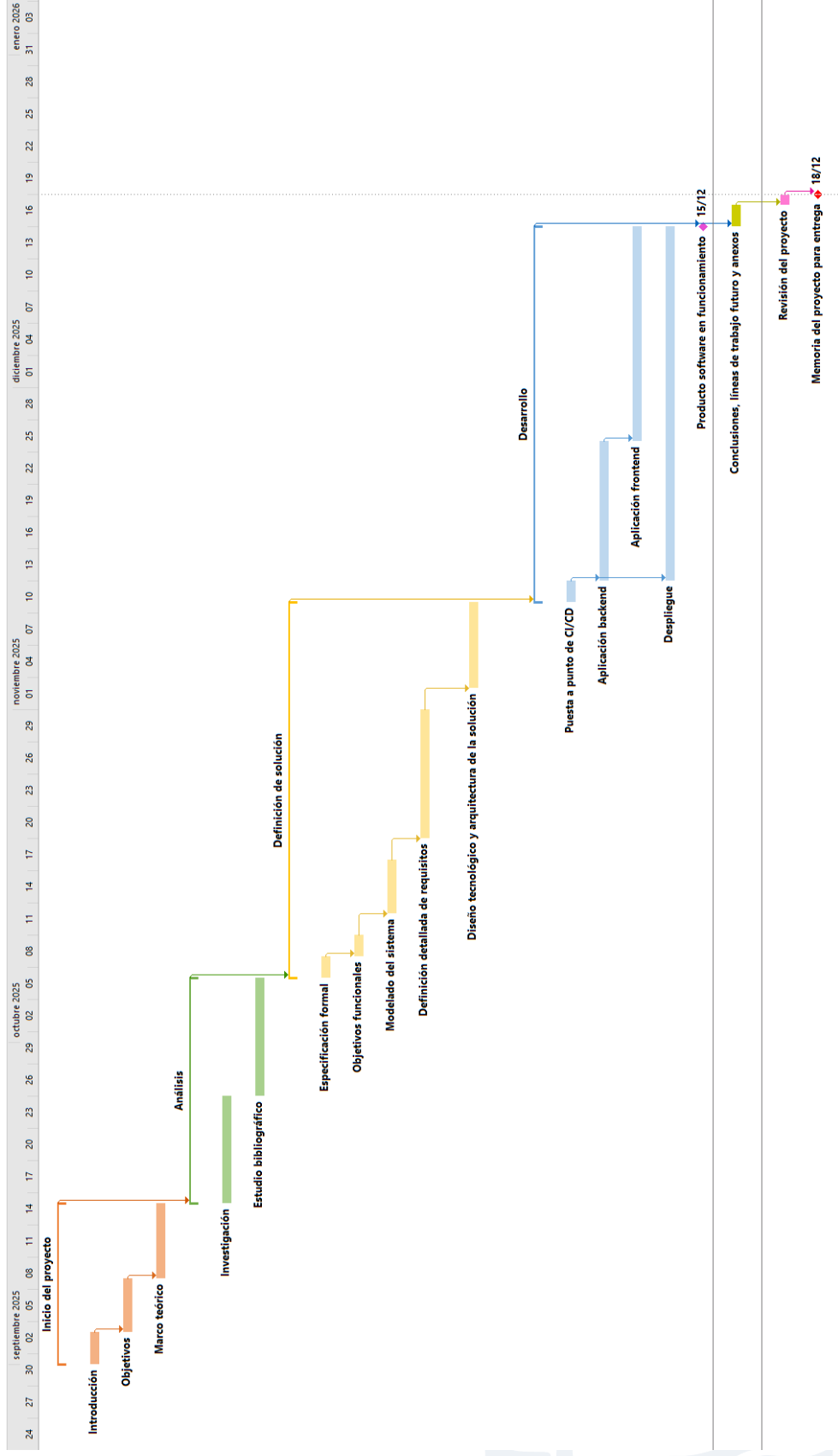


Figura 7.1: Cronograma del proyecto



Por último, también se añade la planificación de las tareas principales en horas.

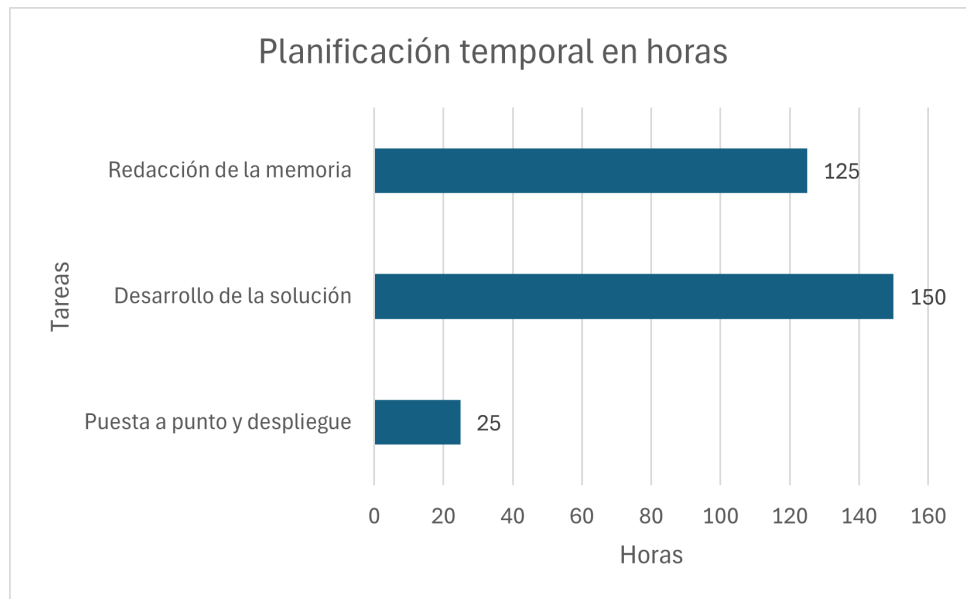


Figura 7.2: Planificación de las tareas principales en horas

8 DESARROLLO DE LA APLICACIÓN BACKEND

En este capítulo se describe el proceso de desarrollo de la aplicación backend del sistema, que actúa como núcleo funcional encargado de gestionar la lógica de negocio, la persistencia de los datos y la comunicación con el frontend y con servicios externos. El backend proporciona una interfaz de acceso uniforme y segura a los recursos del sistema mediante una API REST, permitiendo una interacción desacoplada y escalable entre los distintos componentes de la aplicación.

Se presentan las decisiones de diseño adoptadas, la arquitectura y organización interna del backend, así como los mecanismos implementados para garantizar la seguridad, la calidad y la mantenibilidad del sistema.

8.1 ARQUITECTURA Y VISIÓN GENERAL DEL BACKEND

La aplicación backend se ha diseñado siguiendo una arquitectura cliente-servidor basada en servicios REST, donde el servidor expone un conjunto de endpoints que permiten al frontend interactuar con los distintos recursos del sistema. Las comunicaciones se realizan mediante peticiones HTTP y respuestas en formato JSON, lo que facilita la interoperabilidad y la integración con otros posibles clientes o servicios externos.

Desde el punto de vista estructural, el backend adopta una separación de responsabilidades inspirada en el patrón Modelo-Vista-Controlador (MVC), adaptado al contexto de una API REST. Aunque no existe una capa de vista tradicional, el sistema diferencia claramente entre:

- **Modelos**, responsables de definir la estructura de los datos, sus validaciones y la interacción con la base de datos.
- **Controladores**, que encapsulan la lógica de negocio y coordinan las operaciones sobre los modelos en respuesta a las peticiones recibidas.
- **Rutas**, que actúan como capa de exposición de la API, definiendo los endpoints disponibles y delegando el procesamiento de las peticiones en los controladores correspondientes.

Esta organización favorece un diseño modular y desacoplado, en el que cada componente cumple una función claramente definida. Como resultado, se facilita la mantenibilidad del código, la incorporación de nuevas funcionalidades y la realización de pruebas de forma aislada.

A nivel de proyecto, el backend se estructura en distintos módulos y directorios que reflejan esta separación, incluyendo carpetas específicas para modelos, controladores, rutas y middlewares. La dis-

posición mencionada contribuye a una mayor claridad del código y a una gestión más eficiente del desarrollo, especialmente en proyectos de crecimiento progresivo.

Desde el punto de vista tecnológico, el backend se ha desarrollado utilizando **Node.js** como entorno de ejecución y **Express** como framework principal para la construcción de la API REST. La persistencia de los datos del sistema se gestiona mediante una base de datos documental **MongoDB**, integrada directamente con el backend. Esta capa de persistencia permite almacenar y recuperar de forma eficiente la información relacionada con los distintos elementos del sistema, asegurando la coherencia de los datos y proporcionando una base sólida sobre la que se apoya la lógica de negocio de la aplicación. Los detalles concretos de la modelización y el acceso a los datos se abordan en los apartados posteriores dedicados a la implementación.

El desarrollo y mantenimiento del backend se apoya en un conjunto de herramientas orientadas a la calidad del software, incluyendo pruebas unitarias y de integración, así como un flujo de integración continua que facilita la detección temprana de errores y la estabilidad del sistema.

8.2 MODELADO DE DATOS

Con el objetivo de proporcionar una visión global de la estructura de la información manejada por el sistema, se ha elaborado un modelo conceptual de datos que identifica las principales entidades del dominio y las relaciones existentes entre ellas. Este modelo permite comprender de forma clara cómo se organiza la información dentro de la aplicación y sirve como referencia para la posterior implementación de la persistencia de datos en el backend.

Es importante destacar que el diagrama presentado no corresponde a un diagrama Entidad-Relación (ER) en sentido estricto. Dado que el sistema utiliza una base de datos documental (MongoDB), no resulta adecuado aplicar un modelado relacional tradicional ni imponer las restricciones propias de este tipo de bases de datos. En su lugar, se adopta un enfoque conceptual que refleja la estructura lógica de los datos y sus relaciones semánticas, independientemente de su implementación física.

El modelo se centra, por tanto, en representar las entidades principales del sistema y las asociaciones existentes entre ellas, como relaciones uno a muchos o muchos a muchos sin detallar operaciones. Esta abstracción resulta especialmente adecuada en entornos NoSQL, donde la flexibilidad del esquema permite distintas estrategias de persistencia que se definen a nivel de implementación.

La Figura 8.1 muestra el modelo conceptual de datos del sistema, proporcionando una visión estructural que complementa la definición funcional y arquitectónica presentada en los apartados anteriores.

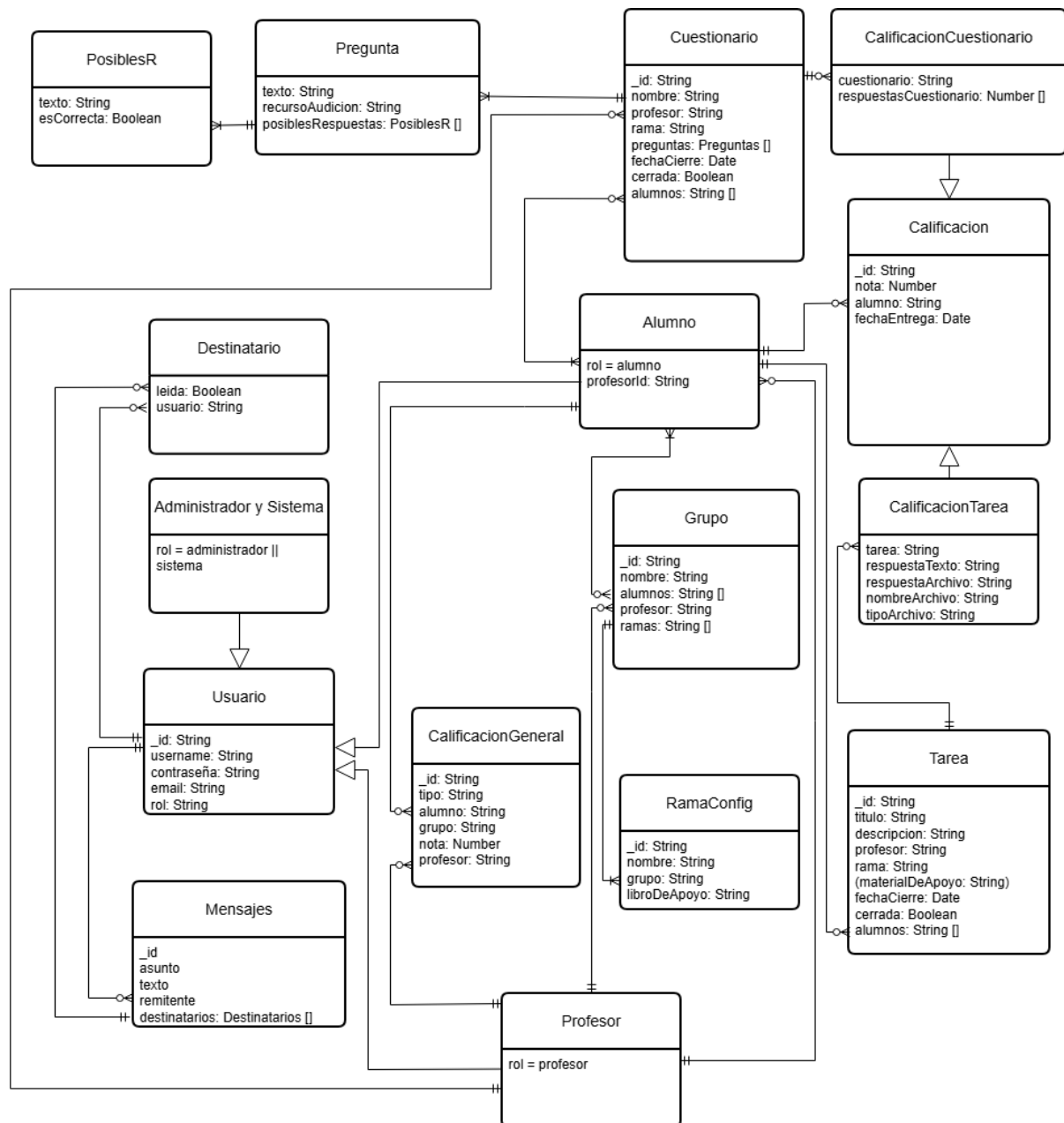


Figura 8.1: Modelo conceptual de datos del sistema

8.3 IMPLEMENTACIÓN DE LA LÓGICA DE NEGOCIO E INTEGRACIÓN

En este apartado se describe la implementación de la lógica de negocio del backend, la integración con servicios externos y los mecanismos de seguridad y documentación que permiten garantizar la calidad, mantenibilidad y escalabilidad del sistema.

8.3.1 Lógica de negocio: creación y actualización de calificaciones generales

Un ejemplo representativo de la lógica de negocio es el método encargado de crear o actualizar calificaciones generales para los alumnos. Este método valida los datos de entrada, controla reglas específicas del dominio (como restricciones para calificaciones extraordinarias), maneja errores específicos y, además, genera un mensaje automático del sistema para notificar al alumno sobre la calificación.

```
exports.crearOActualizarCalificacionGeneral =
async (alumnoId, grupoId, tipo, nota, profesorId) => {
  try {
    const validTypes = ['Q1', 'Q2', 'Q3', 'Ordinaria',
      'Extraordinaria'];
    if (!tipo || !validTypes.includes(tipo)) {
      return { status: 400, body: { error:
        'El tipo de calificación proporcionado no es válido.' } };
    }

    if (!validateNota(nota)) {
      return { status: 400, body: { error:
        'La nota debe ser un número entero entre 1 y 10.' } };
    }

    const alumno = await Usuario.findById(alumnoId);
    if (!alumno || alumno.role !== 'alumno') {
      return { status: 400, body: { error:
        'El ID de alumno proporcionado no es válido
        o el usuario no es un alumno.' } };
    }

    const grupo = await Grupo.findById(grupoId);
    if (!grupo) {
      return { status: 400, body: { error:
        'El ID de grupo proporcionado no es válido.' } };
    }

    if (profesorId) {
      const profesor = await Usuario.findById(profesorId);
      if (!profesor || profesor.role !== 'profesor') {
        return { status: 400, body: { error:
          'El ID de profesor proporcionado no es válido
          o el usuario no es un profesor.' } };
      }
    }

    if (tipo === 'Extraordinaria') {
      const ordinaria = await CalificacionGeneral.findOne
```



```
({ alumno: alumnoId, grupo: grupoId, tipo: 'Ordinaria' });
if (!ordinaria) {
  return { status: 400, body: { error:
    'No se puede establecer una calificación
    Extraordinaria sin una calificación
    Ordinaria previa.' } }};
}
if (ordinaria.nota >= 5) {
  return { status: 400, body: { error:
    'Solo se puede establecer una calificación
    Extraordinaria si la calificación
    Ordinaria es menor de 5.' } }};
}
}

const calificacion = await CalificacionGeneral.findOneAndUpdate(
  { alumno: alumnoId, grupo: grupoId, tipo: tipo },
  { tipo: tipo, nota: nota, profesor: profesorId },
  { upsert: true, new: true, runValidators: true }
);

const sistemaUser =
await Usuario.findOne({ username: 'sistema' });
if (sistemaUser) {
  const asunto = `Calificación general de (${tipo})`;
  const texto = `La calificación obtenida
  en ${tipo} es ${nota}`;
  await mensajeController.
  crearMensaje(sistemaUser._id, asunto, texto, [alumnoId]);
}

return { status: 200, body: calificacion };

} catch (error) {
  if (error.code === 11000) {
    return { status: 409, body: { error:
      'Ya existe una calificación de tipo
      `${tipo}` para este alumno en este grupo.' } }};
  }
  if (error.name === 'ValidationError') {
    return { status: 400, body: { error: error.message } }};
  }
  return { status: 500, body: { error:
    'Error al crear/actualizar la calificación:
    ${error.message}' } }};
```

```

    }
};

```

Este ejemplo muestra el manejo detallado de la lógica y validaciones propias del dominio, la interacción con la base de datos y la generación de mensajes del sistema para mantener informado al usuario.

8.3.2 Integración con servicios externos

Para funcionalidades específicas, como el envío de correos electrónicos, se integran servicios externos que simplifican el desarrollo y aseguran fiabilidad.

Por ejemplo, el siguiente método utiliza la API de Mailjet para enviar credenciales a un usuario nuevo:

```

export const enviarEmailCredenciales =
async (destinatario, username, password) => {
  try {
    const request = mailjetClient
      .post('send', { version: 'v3.1' }).request({
        Messages: [
          {
            From: {
              Email: process.env.EMAIL_FROM,
              Name: process.env.EMAIL_FROM_NAME,
            },
            To: [
              {
                Email: destinatario,
                Name: username,
              },
            ],
            Subject: 'Tus credenciales para SolfEdge',
            HTMLPart: `
<h1>¡Bienvenido a SolfEdge!</h1>
<p>Hola ${username}</p>
<p>Se ha creado una cuenta para ti en SolfEdge.
A continuación encontrarás tus credenciales de acceso:</p>
<ul>
  <li><strong>Usuario:</strong> ${username}</li>
  <li><strong>Contraseña:</strong> ${password}</li>
</ul>
`,
          },
        ],
      });
  } catch (error) {
    console.log(error);
  }
};

```

```
    await request;

    return { message: 'Correo enviado correctamente.' };
  } catch (err) {
    return {
      message: 'Error al enviar el correo.',
      error: err.statusCode ? `${err.statusCode} -` :
        `${err.message}` : err.message || JSON.stringify(err),
    };
  }
};
```

Esta integración permite externalizar tareas complejas, aumentando la robustez y fiabilidad de la aplicación.

8.3.3 Seguridad: autenticación mediante JWT

La seguridad en el acceso a la API se implementa a través de JWT, mediante el cual se genera un token de sesión para cada usuario que se autentica. Este token tendrá caducidad y, tras un periodo determinado (en este caso 1 hora), no podrá seguir siendo utilizado para acceder a recursos protegidos.

```
exports.login = async (username, password) => {
  try {
    const usuario = await Usuario.findOne({ username });
    if (!usuario) {
      return { status: 404, body:
        { error: 'Usuario no encontrado.' } };
    }
    const isMatch = (password === usuario.password);

    if (!isMatch) {
      return { status: 401, body:
        { error: 'Contraseña incorrecta.' } };
    }

    const payload = {
      id: usuario._id,
      username: usuario.username,
      role: usuario.role,
      email: usuario.email
    };

    if (usuario.role === 'alumno') {
      const grupo = await mongoose.model('Grupo').
        findOne({ alumnos: usuario._id });
```

```
        if (grupo) {
            payload.grupoId = grupo._id;
        }
    }

    const token = jwt.sign(
        payload, JWT_SECRET, { expiresIn: '1h' });
    return { status: 200,
        body: { message: 'Login correcto', token: token } };

} catch (error) {
    return { status: 500,
        body: { error: 'Error interno del servidor: ${error.message}' }};
}
};
```

Una vez dentro de la aplicación, el middleware comprobará cada vez que se acceda a una ruta protegida que existe un token válido, en caso contrario, se denegará el acceso.

```
exports.verifyToken = (req, res, next) => {
    if (req.method === 'OPTIONS') {
        return next();
    }

    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1];

    if (!token) {
        return res.status(401).json({ error: 'Acceso denegado.
        No se proporcionó token.' });
    }
    const result = authController.verifySession(token);

    if (result.status === 200) {
        req.user = result.body.sessionData;
        next();
    } else {
        return res.status(result.status).json(result.body);
    }
};
```

De esta forma, todas las rutas relevantes quedan protegidas, permitiendo solo el acceso a usuarios autenticados, salvo aquellas destinadas a usuarios no autenticados y la documentación de la API:

```
app.use('/api/auth', authRoutes);
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
app.use('/api/usuarios', authMiddleware.verifyToken, usuarioRoutes);
app.use('/api/grupos', authMiddleware.verifyToken, grupoRoutes);
app.use('/api/mensajes', authMiddleware.verifyToken, mensajeRoutes);
app.use('/api/ramas', authMiddleware.verifyToken, ramaRoutes);
app.use('/api/tareas', authMiddleware.verifyToken, tareaRoutes);
app.use('/api/cuestionarios',
  authMiddleware.verifyToken, cuestionarioRoutes);
app.use('/api/calificaciones-generales',
  authMiddleware.verifyToken, calificacionGeneralRoutes);
app.use('/api/calificaciones',
  authMiddleware.verifyToken, calificacionRoutes);
```

8.3.4 Documentación de la API con Swagger

Para facilitar la colaboración, mantenimiento y pruebas, la API está documentada con **Swagger**, que genera automáticamente una interfaz web interactiva accesible en la ruta `/api-docs`.

Esta documentación permite explorar los endpoints disponibles, con detalles sobre métodos, parámetros y respuestas, manteniendo la información siempre actualizada gracias a la sincronización con el código fuente.

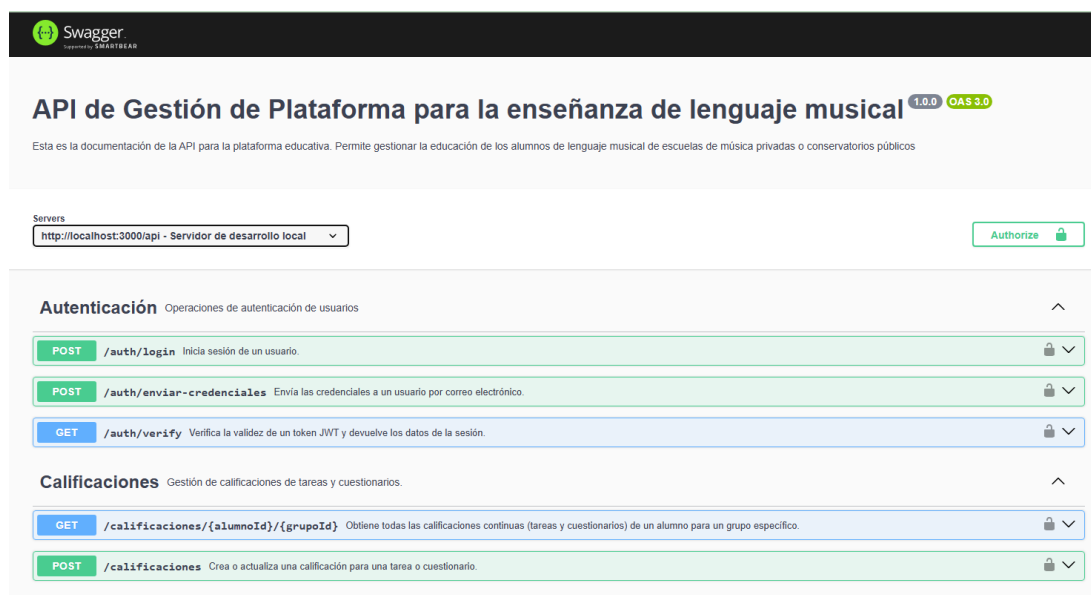


Figura 8.2: Interfaz de la documentación generada por Swagger

El enfoque dado mejora la mantenibilidad del sistema y simplifica la integración de nuevos desarrolladores o testers durante el ciclo de vida del proyecto.

Con esta estructura, la implementación del backend cubre desde la lógica de negocio hasta la seguridad y la documentación, garantizando un sistema robusto, seguro y fácil de mantener.

8.4 TESTING

Para asegurar la calidad y fiabilidad del backend, se ha implementado un conjunto de pruebas automatizadas que abarcan diferentes niveles, con el objetivo de detectar errores tempranamente y facilitar el mantenimiento del código.

8.4.1 Estrategia de pruebas

El proceso de testing se basa en dos tipos principales de pruebas:

- **Pruebas unitarias:** Se validan individualmente las funciones y métodos que componen la lógica de negocio y el acceso a datos, comprobando que cada unidad cumpla con el comportamiento esperado ante diversas situaciones.
- **Pruebas de integración:** Se verifican las interacciones entre distintos módulos y componentes del backend, incluyendo la comunicación con la base de datos y la integración con servicios externos, asegurando que el sistema funcione correctamente como conjunto.

8.4.2 Herramientas utilizadas

Se han utilizado herramientas especializadas que permiten automatizar y gestionar de forma eficiente las pruebas:

- **Jest:** Framework para pruebas unitarias en JavaScript, que proporciona funcionalidades para mocks, aserciones y cobertura de código.
- **Supertest:** Herramienta para realizar pruebas sobre endpoints HTTP, facilitando la simulación de peticiones y validación de respuestas en la API REST.

8.4.3 Ejemplo de prueba

A modo ilustrativo, se presenta un ejemplo de prueba unitaria sobre la función que crea o actualiza una calificación general, comprobando tanto el resultado esperado como el manejo de errores:

```
describe('POST /calificaciones-generales', () => {
  it('should create or update a general qualification',
    async () => {
      const mockCalificacion = {
        _id: 'calG1',
        alumno: 'alumno1',
        grupo: 'grupo1',
        tipo: 'Ordinaria',
        nota: 8,
        toObject: () => ({
          _id: 'calG1',
          alumno: 'alumno1',
          grupo: 'grupo1',
          tipo: 'Ordinaria',
          nota: 8
        })
      }

      Usuario.findById
        .mockResolvedValueOnce({ _id: 'alumno1',
          role: 'alumno' })
        .mockResolvedValueOnce({ _id: 'profesor1',
          role: 'profesor' });

      Grupo.findById.mockResolvedValue
        ({ _id: 'grupo1' });
      CalificacionGeneral.findOneAndUpdate.
        mockResolvedValue(mockCalificacion);
      Usuario.findOne.mockResolvedValue
        ({ _id: 'sistema-user' });
      mensajeController.crearMensaje.
        mockResolvedValue({});

      const response = await request(app)
        .post('/calificaciones-generales')
        .send({
          alumnoId: 'alumno1',
          grupoId: 'grupo1',
          tipo: 'Ordinaria',
          nota: 8,
          profesorId: 'profesor1'
        })
    })
});
```

```
    });

    expect(response.status).toBe(200);
    expect(response.body).toEqual(
      (mockCalificacion.toObject());
    });
  });
  it('should return 400 for invalid nota', async () => {
    const response = await request(app)
      .post('/calificaciones-generales')
      .send({ alumnoId: 'alumno1',
        grupoId: 'grupo1', tipo: 'Ordinaria',
        nota: 11, profesorId: 'profesor1' });

    expect(response.status).toBe(400);
    expect(response.body).toEqual(
      ({ error: 'La nota debe ser un número entero
        entre 1 y 10.' }));
  });
  it('should return 400 if Extraordinaria is set
    without prior Ordinaria', async () => {
    Usuario.findById
      .mockResolvedValueOnce
      ({ _id: 'alumno1', role: 'alumno' })
      .mockResolvedValueOnce
      ({ _id: 'profesor1', role: 'profesor' });

    Grupo.findById.mockResolvedValue({ _id: 'grupo1' });
    CalificacionGeneral.findOne.mockResolvedValue(null);

    const response = await request(app)
      .post('/calificaciones-generales')
      .send({
        alumnoId: 'alumno1',
        grupoId: 'grupo1',
        tipo: 'Extraordinaria',
        nota: 6,
        profesorId: 'profesor1'
      });

    expect(response.status).toBe(400);
    expect(response.body).toEqual({
      error: 'No se puede establecer una
        calificación Extraordinaria
        sin una calificación Ordinaria previa.'
    });
  });
});
```




```
it('should return 400 if Extraordinaria is
set with Ordinaria >= 5', async () => {
  Usuario.findById
    .mockResolvedValueOnce
    ({ _id: 'alumno1', role: 'alumno' })
    .mockResolvedValueOnce
    ({ _id: 'profesor1', role: 'profesor' });

  Grupo.findById.mockResolvedValue
    ({ _id: 'grupo1' });
  CalificacionGeneral.findOne.mockResolvedValue
    ({ nota: 5 });

  const response = await request(app)
    .post('/calificaciones-generales')
    .send({
      alumnoId: 'alumno1',
      grupoId: 'grupo1',
      tipo: 'Extraordinaria',
      nota: 6,
      profesorId: 'profesor1'
    });

  expect(response.status).toBe(400);
  expect(response.body).toEqual({
    error: 'Solo se puede establecer una calificación
    Extraordinaria si la calificación Ordinaria
    es menor de 5.'
  });
});
});
```

8.5 DESPLIEGUE DEL BACKEND

El despliegue de la aplicación backend se realiza mediante un flujo automatizado de integración y despliegue continuo (CI/CD) que garantiza calidad y rapidez en las entregas.

Se utiliza **GitHub Actions** para orquestar el pipeline de integración, que incluye los siguientes pasos:

- **Compilación:** Se verifica que el código fuente compile correctamente y cumpla con las configuraciones definidas. Este paso es el único ejecutado en el pipeline de GitHub Actions para el backend.

- **Ejecución de pruebas:** Las pruebas unitarias y de integración se ejecutan localmente o en otros pipelines especializados para validar la funcionalidad, en este caso, sí que se ejecutarán en el build principal del backend para evitar desplegar la aplicación con comportamiento incorrecto.
- **Despliegue automático:** El despliegue en producción se realiza mediante **Render**, que monitoriza el repositorio y actualiza la aplicación automáticamente con cada commit. Render cachea los artefactos y dependencias para acelerar los despliegues futuros.

Este proceso aporta varios beneficios, entre ellos:

- **Calidad constante:** Al integrar la compilación en el pipeline se asegura que solo código que compile correctamente avance hacia producción.
- **Agilidad en las entregas:** La automatización del despliegue en Render reduce tiempos y riesgos asociados a despliegues manuales, facilitando iteraciones rápidas y confiables.
- **Transparencia y trazabilidad:** Render y GitHub Actions registran cada paso del proceso, facilitando la detección y solución de incidencias.

✓ fix backend test corregido #107



The screenshot shows the GitHub Actions interface for a workflow named 'fix backend test corregido #107'. On the left, a sidebar lists jobs: 'build-frontend', 'build-backend' (selected), and 'deploy-frontend'. The main panel shows the 'build-backend' job, which succeeded 4 minutes ago in 32s. The job steps are listed as follows:

- Set up job
- Checkout code
- Set up Nodejs
- Install dependencies
- Run backend tests (Jest)
- Post Set up Nodejs
- Post Checkout code
- Complete job

On the right, a detailed view of the 'Run backend tests (Jest)' step shows the following output:

```

45 Test Suites: 10 passed, 10 total
46 Tests:      137 passed, 137 total
47 Snapshots:  0 total
48 Time:       10.708 s
49 Ran all test suites.
  
```

(a) Paso de compilación del backend en GitHub Actions

(b) Detalle de la ejecución de pruebas unitarias e integración

Figura 8.3: Pipeline de integración continua: Backend

Este enfoque mantiene el backend robusto y actualizado, alineado con las buenas prácticas actuales de desarrollo y despliegue de software.

9 DESARROLLO DE LA APLICACIÓN FRONTEND

En este capítulo se describe el proceso de desarrollo de la aplicación frontend del sistema, que actúa como interfaz de usuario y proporciona la experiencia visual e interactiva para la gestión y uso de las funcionalidades ofrecidas por el backend. La aplicación frontend se encarga de presentar la información, gestionar la interacción del usuario y coordinar las llamadas a la API para obtener o enviar datos.

Se presentan las decisiones de diseño adoptadas, la arquitectura y organización interna del frontend, así como las tecnologías utilizadas para garantizar una experiencia de usuario eficiente, intuitiva y mantenible. La aplicación frontend se ha desarrollado como una aplicación híbrida utilizando **Ionic** y **Angular**, lo que permite construir una única base de código capaz de ejecutarse tanto en entornos web como en dispositivos móviles. Esta aproximación facilita el desarrollo multiplataforma y garantiza una experiencia de usuario coherente independientemente del dispositivo utilizado.

Angular se emplea como framework principal para la estructuración de la aplicación, el manejo de componentes, la navegación y la gestión del estado, mientras que Ionic proporciona un conjunto de componentes visuales y herramientas orientadas a la creación de interfaces modernas, responsivas y adaptadas a dispositivos móviles.

La comunicación con el backend se realiza mediante peticiones HTTP a una API REST, integrando mecanismos de autenticación basados en JWT y una gestión centralizada de servicios para el acceso a los datos. El proceso de desarrollo se apoya en las herramientas propias del ecosistema Angular para la construcción, optimización y prueba de la aplicación.

9.1 ARQUITECTURA Y VISIÓN GENERAL DEL FRONTEND

La aplicación frontend está construida siguiendo una arquitectura en el patrón Modelo-Vista-Controlador (MVC), que facilita una clara separación de responsabilidades y mejora la escalabilidad y mantenibilidad del código. A diferencia del backend, donde el patrón MVC está adaptado a un contexto API sin capa de vista tradicional, en el frontend este patrón se implementa en toda su extensión, permitiendo gestionar de forma estructurada la presentación, la lógica de interacción y el manejo de datos.

En concreto, la aplicación se organiza en tres componentes fundamentales:

- **Modelo**, que representa la estructura y estado de los datos gestionados en la interfaz, incluyendo la gestión del estado local y global, así como la integración con las respuestas del backend.

- **Vista**, que comprende los elementos visuales y la interfaz de usuario, diseñada para mostrar la información de forma clara y facilitar la interacción con el sistema. Esta capa incluye componentes visuales reutilizables, plantillas y estilos que conforman la experiencia gráfica.
- **Controlador**, responsable de mediar entre la vista y el modelo, gestionando la lógica de negocio del frontend, el manejo de eventos, la validación de datos y las comunicaciones con el backend mediante llamadas a la API.

Esta estructura modular permite un desarrollo más organizado, donde cada capa tiene una función bien delimitada, facilitando la colaboración entre equipos, la incorporación de nuevas funcionalidades y la detección y corrección de errores.

A nivel de proyecto, el frontend se organiza en carpetas y módulos que reflejan esta división, agrupando componentes, servicios, modelos y utilidades. Esta disposición contribuye a mantener un código limpio y escalable, preparado para el crecimiento y evolución futura de la aplicación.

Las tecnologías empleadas para el desarrollo de la interfaz se eligieron con el objetivo de maximizar la usabilidad y el rendimiento, incluyendo frameworks modernos de JavaScript, sistemas de gestión de estado y herramientas de construcción y optimización.

En los apartados siguientes se detallan la implementación de estos conceptos, la estructura del código y la forma en que se ha construido la experiencia visual para el usuario final.

9.2 IMPLEMENTACIÓN

En este apartado se describe la implementación técnica de la aplicación frontend, abordando la organización del proyecto, los principales componentes y vistas, así como los mecanismos empleados para la gestión del estado, la navegación, la validación de datos y la integración con el backend. El objetivo principal ha sido construir una aplicación modular, mantenible y coherente con la arquitectura definida en el apartado anterior.

9.2.1 Estructura del proyecto

El proyecto frontend, al haber sido desarrollado en Ionic sobre Angular, permite crear una aplicación híbrida basada en componentes modulares, preparada para su ejecución en entornos web y móviles.

La organización del código sigue las convenciones de Angular y se estructura en carpetas principales:

- **Pages**: Contienen las vistas completas que corresponden a rutas concretas de la aplicación. Cada página combina una plantilla HTML, estilos y una clase TypeScript que actúa como controlador de la vista.
- **Components**: Incluyen componentes reutilizables, que se integran dentro de las páginas para construir interfaces complejas de forma modular.

- **Services:** Agrupan la lógica de negocio, la gestión del estado y la comunicación con el backend a través de llamadas HTTP.
- **Guards:** Implementan la protección de rutas, restringiendo el acceso en función del estado de autenticación y permisos del usuario.
- **Models:** Definen las estructuras de datos utilizadas en la aplicación, facilitando la tipificación y validación de la información manejada.

Esta disposición favorece la separación de responsabilidades, mejora la escalabilidad y facilita el mantenimiento.

9.2.2 Páginas, componentes y controladores

En la aplicación frontend basada en Ionic y Angular, cada vista o página se implementa como un *componente standalone*, lo que permite declarar todas sus dependencias de forma local y favorece la modularidad y el rendimiento de la aplicación. Este enfoque aprovecha las características modernas de Angular para reducir la necesidad de módulos globales y facilita la reutilización y el mantenimiento.

A continuación se muestra un fragmento representativo del controlador de una página Angular implementada como componente standalone, que ilustra la combinación de importaciones necesarias, la inyección de servicios, y el manejo de la lógica de interacción mediante métodos y suscripciones a observables:

```
import { Component, inject, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { forkJoin } from 'rxjs';
import { AuthService } from '../services/auth.service';
import { CalificacionService } from '../services/calificacion.service';
import { CalificacionGeneralService }
from '../services/calificacion-general.service';
import { Usuario } from '../models/usuario.model';
import { PerfilCalificacion }
from '../models/perfil-calificacion.model';
import { CalificacionGeneral }
from '../models/calificacionGeneral.model';
import { FormsModule } from '@angular/forms';
import {
  IonButtons, IonMenuButton, IonHeader, IonToolbar, IonTitle,
  IonContent, IonCard, IonCardHeader, IonCardTitle, IonCardContent,
  IonItem, IonLabel, IonText, IonList, IonSpinner, IonSegment,
  IonSegmentButton, IonListHeader, IonNote, IonButton, IonIcon,
  ToastController, ModalController
} from '@ionic/angular/standalone';
import { MensajeService } from '../services/mensaje.service';
```



```
import { Mensaje } from '../models/mensaje.model';
import { ribbonOutline, sendOutline } from 'ionicons/icons';
import { addIcons } from 'ionicons';
import { GrupoStateService } from '../services/grupo-state.service';
import { Grupo } from '../models/grupo.model';
import { MensajeModalComponent }
from '../components/mensaje-modal/mensaje-modal.component';
import { CalificacionGeneralModalComponent } from
'../components/calificacion-general-modal
/calificacion-general-modal.component';
import { ActivatedRoute, Router } from '@angular/router';

@Component({
  selector: 'app-tab5',
  templateUrl: 'tab5.page.html',
  styleUrls: ['tab5.page.scss'],
  standalone: true,
  imports: [
    CommonModule, FormsModule, IonButtons, IonMenuButton, IonHeader,
    IonToolbar, IonTitle, IonContent, IonCard, IonCardHeader,
    IonCardTitle, IonCardContent, IonItem, IonLabel, IonText,
    IonList, IonSpinner, IonSegment, IonSegmentButton,
    IonListHeader, IonNote, IonButton, IonIcon
  ]
})
export class Tab5Page implements OnInit {
  activatedRoute: ActivatedRoute = inject(ActivatedRoute);
  router: Router = inject(Router);
  authService: AuthService = inject(AuthService);
  calificacionService: CalificacionService =
  inject(CalificacionService);
  calificacionGeneralService: CalificacionGeneralService =
  inject(CalificacionGeneralService);
  mensajeService: MensajeService = inject(MensajeService);
  grupoService: GrupoStateService = inject(GrupoStateService);
  toastController: ToastController = inject(ToastController);
  modalController: ModalController = inject(ModalController);

  selectedGrupo: Grupo | null = null;
  mensajes: Mensaje[] = [];
  currentUser: Usuario | null = null;
  calificacionesContinuas: PerfilCalificacion[] = [];
  calificacionesGeneralesList: CalificacionGeneral[] = [];
  isLoading: boolean = true;
  segmentoSeleccionado: string = 'continuas';
```



```
constructor() {
  addIcons({
    'ribbon-outline': ribbonOutline,
    'send-outline': sendOutline
  });
}

ngOnInit() {
  this.authService.currentUser.subscribe(user => {
    this.currentUser = user;
    this.grupoService.selectedGrupo$.subscribe(grupo => {
      if (grupo) {
        this.selectedGrupo = grupo;
        if (user?.role === 'alumno') {
          this.loadAllGrades(user!._id, grupo._id);
        }
        this.isLoading = false;
        this.activatedRoute.queryParams.subscribe(params => {
          const tool = params['tool'];
          if (tool) {
            if (tool === 'mensajes') {
              this.presentMensajeModal();
            } else if (tool === 'calificaciones') {
              this.presentCalificacionGeneralModal();
            }
            this.router.navigate([], {
              relativeTo: this.activatedRoute,
              queryParams: { tool: null },
              queryParamsHandling: 'merge'
            });
          }
        });
      }
    });
  });
}

async presentCalificacionGeneralModal() {
  if (!this.selectedGrupo) {
    await this.presentToast('Por favor, selecciona un grupo primero.', 'warning');
    return;
  }
  if (!this.currentUser!._id) {
    await this.presentToast('Error: ID de profesor no disponible para calificar.', 'danger');
  }
}
```

```
        return;
    }

    const modal = await this.modalController.create({
        component: CalificacionGeneralModalComponent,
        componentProps: {
            grupoId: this.selectedGrupo._id
        }
    });
    modal.present();

    const { data, role } = await modal.onWillDismiss();

    if (role === 'confirm' && data) {
        this.calificacionGeneralService.
            crearOActualizarCalificacion(
                data.alumnoId,
                this.selectedGrupo._id,
                data.selectedTipo,
                data.nota,
                this.currentUser!._id
            ).subscribe({
                next: async (calificacion) => {
                    await this.presentToast
                        ('Calificación '${calificacion.tipo}'
                         guardada para ${data.alumnoUsername}.');
                },
                error: async (err) => {
                    console.error('Error al guardar
                        calificación general:', err);
                    const errorMessage = err.error && err.error.error
                        ? err.error.error : 'Error al guardar la calificación general.';
                    await this.presentToast(errorMessage, 'danger');
                }
            });
    }
}

// Métodos auxiliares como presentToast,
// loadAllGrades, presentMensajeModal, etc.
// se implementan para manejar la UI y
// comunicación con servicios.
}
```


Aspectos de la implementación:

- **Standalone Components:** Se emplea la nueva funcionalidad de Angular para definir componentes independientes, declarando explícitamente en la propiedad `imports` todos los módulos y componentes que requiere esta página (como módulos comunes, formularios y componentes Ionic). Esto simplifica la carga y mejora la modularidad.
- **Inyección de dependencias:** Se utiliza la función `inject()` para obtener instancias de servicios y otros elementos Angular como `Router` o `ActivatedRoute`, promoviendo un código más limpio y fácil de testear.
- **Ciclo de vida:** El método `ngOnInit()` se encarga de ejecutarse siempre al iniciar el componente, pertenece al conjunto de métodos del ciclo de vida de Angular (`ngOnInit`, `ngAfterViewInit`, `ngOnDestroy`...) y en él se establecen las suscripciones a observables que gestionan el estado del usuario, el grupo seleccionado y parámetros de ruta, permitiendo reaccionar dinámicamente a cambios en el entorno de ejecución.
- **Ionic Components y UX:** La interfaz visual está construida con componentes híbridos de Ionic que funcionan perfectamente tanto en web como en dispositivos móviles (botones, listas, tarjetas, segment buttons, iconos). Para los iconos, se emplea la librería `Ionicons` y se añaden mediante `addIcons()`.
- **Gestión de modales:** El uso de `ModalController` permite abrir componentes como modales para interacción más compleja sin abandonar la página actual. En el ejemplo, la gestión de calificaciones se hace a través de un modal, que tras cerrarse devuelve datos y desencadena acciones en la página padre.
- **Reactividad y asincronía:** Se manejan eventos asíncronos mediante observables y promesas, facilitando la comunicación fluida con servicios externos y actualizando la interfaz de forma reactiva.

Este ejemplo sintetiza cómo combinar de forma eficiente las capacidades de Angular e Ionic para construir una aplicación frontend robusta, modular y con buena experiencia de usuario, aprovechando al máximo las buenas prácticas modernas de desarrollo.

9.2.3 Manejo de rutas y navegación

La navegación entre vistas se gestiona mediante el sistema de enrutado de Angular integrado con Ionic. Las rutas permiten definir qué componente se carga en función de la URL y facilitan la protección de vistas que requieren autenticación.

A modo de ejemplo, el siguiente fragmento muestra la protección de rutas mediante un *guard* de autenticación que tiene como misión la gestión de rutas en función de los roles y la verificación del token JWT que, como se ha mencionado anteriormente, se almacena en el frontend tras el login y protege el acceso a la información desde la interfaz ya que se comprueban las validaciones que el backend realiza en cada petición:

```
export class AdminGuard implements CanActivate {

  constructor(private authService: AuthService, private router: Router)
  {}

  canActivate(): boolean {
    const currentUser = this.authService.currentUserValue;
    if (this.authService.isAuthenticated()) {
      if (currentUser && currentUser.role === 'administrador') {
        return true;
      } else {
        this.router.navigate(['/Areas']);
        return false;
      }
    } else {
      this.router.navigate(['/Login']);
      return false;
    }
  }
}
```

De este modo, solo los usuarios autenticados pueden acceder a la aplicación y ciertas rutas quedan restringidas según el rol del usuario, mejorando la seguridad y experiencia de navegación.

9.2.4 Gestión del estado

La gestión del estado del frontend se realiza principalmente mediante servicios de Angular que actúan como fuente centralizada de información compartida entre los distintos componentes. Este enfoque permite mantener la coherencia de los datos durante la sesión del usuario y evitar duplicaciones innecesarias.

Un ejemplo representativo es el servicio de autenticación, encargado de almacenar y proporcionar la información del usuario autenticado y su token JWT:

```
import { Injectable } from '@angular/core';
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { environment } from '../../environments/environment';
import { tap } from 'rxjs/operators';
import { Observable, BehaviorSubject } from 'rxjs';
import { Usuario } from '../models/usuario.model';

@Injectable({
  providedIn: 'root'
})
```



```
export class AuthService {
  private apiUrl = environment.apiUrl;
  private readonly TOKEN_KEY = 'auth-token';

  private currentUserSubject: BehaviorSubject<Usuario | null>;
  public currentUser: Observable<Usuario | null>;

  constructor(private http: HttpClient) {
    this.currentUserSubject =
      new BehaviorSubject<Usuario | null>(this.getUserFromStorage());
    this.currentUser = this.currentUserSubject.asObservable();
  }

  login(username: string, password: string):
  Observable<{token: string}> {
    return this.http.post<{token: string}>
      ('${this.apiUrl}/auth/login', { username, password }).pipe(
        tap(response => {
          localStorage.setItem(this.TOKEN_KEY, response.token);
        })
      );
  }

  verifyAndFetchUser(): Observable<any> {
    return this.http.get<any>(`${this.apiUrl}/auth/verify`).pipe(
      tap(response => {
        if (response && response.sessionData) {
          const user: Usuario = {
            _id: response.sessionData.id,
            username: response.sessionData.username,
            role: response.sessionData.role,
            email: response.sessionData.email,
            groupId: response.sessionData.groupId
          };
          localStorage.setItem('currentUser', JSON.stringify(user));
          this.currentUserSubject.next(user);
        }
      })
    );
  }

  logout(): void {
    localStorage.removeItem(this.TOKEN_KEY);
    localStorage.removeItem('currentUser');
    this.currentUserSubject.next(null);
  }
}
```

```
getToken(): string | null {  
    return localStorage.getItem(this.TOKEN_KEY);  
}  
  
isAuthenticated(): boolean {  
    return !!this.getToken();  
}  
  
private getUserFromStorage(): Usuario | null {  
    const user = localStorage.getItem('currentUser');  
    return user ? JSON.parse(user) : null;  
}  
  
public get currentUserValue(): Usuario | null {  
    return this.currentUserSubject.value;  
}  
}
```

Este servicio permite que distintos componentes y guards consulten el estado de autenticación sin depender directamente entre sí.

9.2.5 Integración con el backend

La comunicación con el backend se realiza mediante servicios que encapsulan las llamadas HTTP a la API REST. Este diseño desacopla la lógica de comunicación del resto de la aplicación y facilita su reutilización y mantenimiento.

A continuación se muestra un fragmento simplificado de un servicio que realiza una petición autenticada al backend:

```
crearOActualizarCalificacion(  
    alumnoId: string,  
    grupoId: string,  
    tipo: 'Q1' | 'Q2' | 'Q3' | 'Ordinaria' | 'Extraordinaria',  
    nota: number,  
    profesorId?: string  
): Observable<CalificacionGeneral> {  
    const body = { alumnoId, grupoId, tipo, nota, profesorId };  
    return this.http.post<CalificacionGeneral>(this.apiUrl, body);  
}
```

Las respuestas del backend se procesan mediante observables, permitiendo una gestión eficiente de los datos y de los posibles errores.

9.3 DISEÑO Y EXPERIENCIA DE USUARIO (UX/UI)

El diseño de la aplicación frontend se ha concebido para ofrecer una experiencia visual y funcional óptima, que sea adaptable a múltiples dispositivos, intuitiva para todo tipo de usuarios y accesible conforme a las mejores prácticas de usabilidad.

Gracias a la utilización del framework **Ionic** combinado con **Angular**, la aplicación aprovecha una amplia colección de componentes híbridos que aseguran un comportamiento nativo y coherente tanto en plataformas móviles (iOS, Android) como en navegadores web. Esto permite un desarrollo ágil sin sacrificar la calidad del diseño ni la experiencia del usuario.

9.3.1 Componentes Ionic y su papel en el diseño

La aplicación hace uso extensivo de la gran variedad de componentes que ofrece Ionic, permitiendo construir interfaces sofisticadas sin perder simplicidad. Algunos componentes clave y su función en la aplicación son:

- **IonHeader, IonToolbar y IonTitle:** Para estructuras de cabecera consistentes que mantienen el branding y navegación principal.
- **IonList y IonItem:** Presentan colecciones de datos, permitiendo selección, búsqueda y acciones contextualizadas.
- **IonSegment e IonSegmentButton:** Segmentan contenido y permiten filtros rápidos sin recargar la página.
- **IonModal:** Facilita interacciones superpuestas que no interrumpen el flujo principal.
- **IonSpinner y IonToast:** Informan de estados de carga y notificaciones breves.
- **IonButtons, IonMenuButton y IonTabs:** Gestionan la navegación adaptable y accesible en diferentes dispositivos.
- **IonIcon:** Mejora la comunicación visual usando iconografía clara y estilizada.
- **IonInput, IonTextarea, IonSelect:** Para formularios dinámicos y accesibles con validación integrada.
- **IonGrid, IonRow y IonCol:** Implementan layouts flexibles para distribuir el contenido responsivamente.

Cada uno de estos componentes se ha configurado y estilizado para ajustarse a la identidad gráfica del proyecto, sin perder las ventajas nativas que ofrece Ionic en términos de rendimiento y adaptabilidad.

9.3.2 Diseño responsive y adaptabilidad

La naturaleza híbrida de Ionic facilita que la aplicación se adapte automáticamente a diferentes tamaños y orientaciones de pantalla, desde smartphones y tablets hasta ordenadores de escritorio. Este diseño responsivo no depende de hojas de estilo CSS personalizadas con media queries, sino que se basa en:

- **Componentes y layouts flexibles:** Ionic provee componentes como `IonGrid`, `IonRow` y `IonCol` que implementan un sistema de rejilla basado en flexbox, permitiendo distribuir el contenido de forma fluida y proporcional según el espacio disponible.
- **Controles de tamaño y espaciado adaptativos:** Los componentes ajustan automáticamente márgenes, padding y tamaños internos para asegurar legibilidad y usabilidad, evitando que elementos se amontonen o queden demasiado dispersos.
- **Optimización para gestos táctiles y clics:** Los botones, listas y controles interactivos están dimensionados y espaciados para ser fácilmente seleccionables tanto con el dedo como con el ratón.
- **Navegación adaptable:** El menú lateral se muestra como panel desplegable en móvil y como barra fija en escritorio, mientras que la barra de pestañas permite un acceso rápido a funcionalidades principales sin saturar la interfaz.

Por ejemplo, en la Figura 9.1 se puede observar cómo la pantalla de inicio de sesión presenta un formulario centrado con tamaño óptimo en móvil y escritorio, sin necesidad de adaptar manualmente estilos CSS específicos.

Esta adaptabilidad se traduce en una experiencia coherente, sin necesidad de crear versiones separadas de la aplicación para cada plataforma.

9.3.3 Usabilidad y experiencia de usuario

El diseño prioriza la facilidad de uso, minimizando la curva de aprendizaje y facilitando tareas comunes con patrones visuales y de interacción familiares. Algunos aspectos clave son:

- **Estructura clara y consistente:** La agrupación lógica de funciones mediante menús, pestañas y componentes visibles evita confusión y reduce la carga cognitiva.
- **Feedback visual inmediato:** Elementos como botones muestran estados activos, inactivos o de carga (por ejemplo, usando `IonSpinner`), informando al usuario sobre el estado de la aplicación.
- **Componentes reutilizables:** El uso de componentes modulares (cards, listas, formularios, modales) garantiza uniformidad y facilita la interacción predecible en toda la aplicación.
- **Gestión eficaz de errores y validación:** Los formularios y controles incorporan mensajes claros de validación y errores, guiando al usuario para corregir entradas y evitando frustraciones.

- **Interacciones enriquecidas:** Se emplean modales, segmentos y notificaciones (toasts) para facilitar flujos de trabajo sin cambiar de pantalla o interrumpir tareas.

Un buen ejemplo de usabilidad se aprecia en la gestión de calificaciones donde, a través de un `IonModal` (ver Figura 9.3), el usuario puede editar datos sin salir de la vista principal, manteniendo el contexto y agilizando la interacción. Otro buen ejemplo es el uso de distintos menús según el contexto (Figura 9.2).

9.3.4 Accesibilidad

Más allá de la simple apariencia, la aplicación se ha diseñado para ser accesible, garantizando que personas con diversas capacidades puedan utilizarla eficazmente. Esto se logra mediante:

- **Soporte ARIA y roles semánticos:** Ionic incorpora atributos ARIA en sus componentes, mejorando la compatibilidad con lectores de pantalla y tecnologías asistivas.
- **Contraste y legibilidad:** La paleta de colores se ha escogido para ofrecer un contraste adecuado entre texto, fondos y elementos interactivos, facilitando la lectura incluso en condiciones de baja iluminación o para personas con deficiencias visuales (Figura 9.4).
- **Etiquetado claro y descriptivo:** Los formularios, botones y controles cuentan con etiquetas y descripciones textuales que clarifican su función y estado.

El compromiso con la accesibilidad se refleja en la mejora de la experiencia global, haciendo que la aplicación sea usable por un público más amplio y cumpliendo con estándares internacionales.

9.3.5 Ejemplos visuales de diseño y experiencia

Estas imágenes reflejan cómo la combinación de Ionic y Angular junto con buenas prácticas de diseño se traduce en una aplicación usable, accesible y responsiva, lista para su despliegue en múltiples plataformas y entornos de usuario.



Figura 9.1: Vista de inicio de sesión en móvil y escritorio

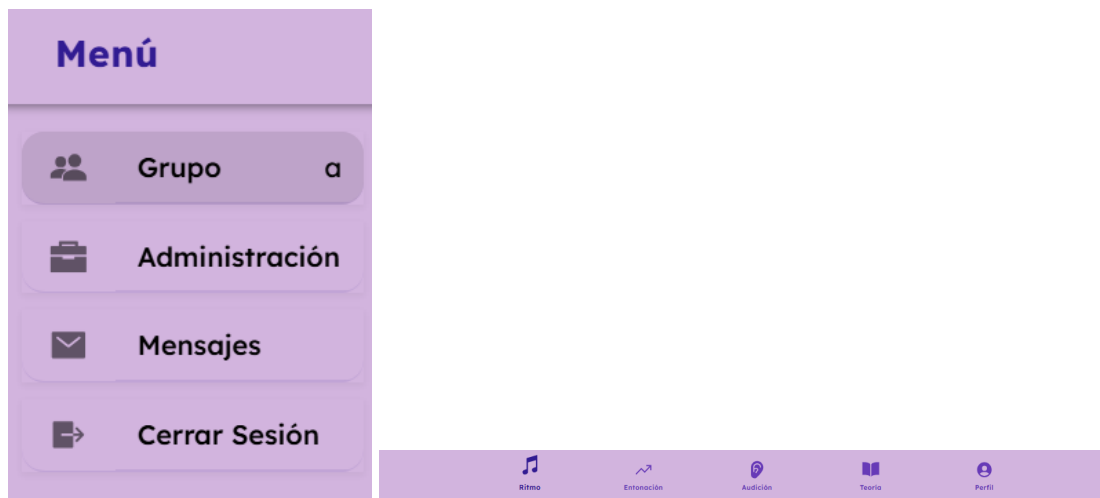


Figura 9.2: Menú lateral y barra de pestañas.

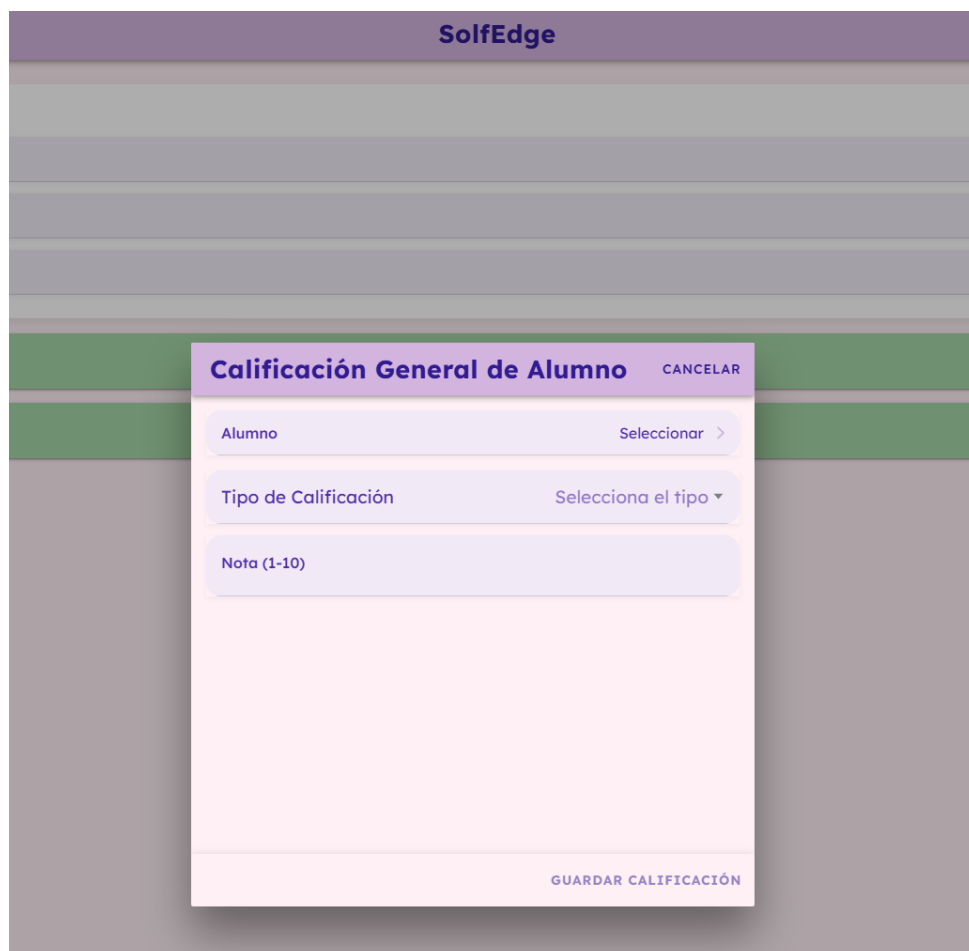


Figura 9.3: Uso de IonModal

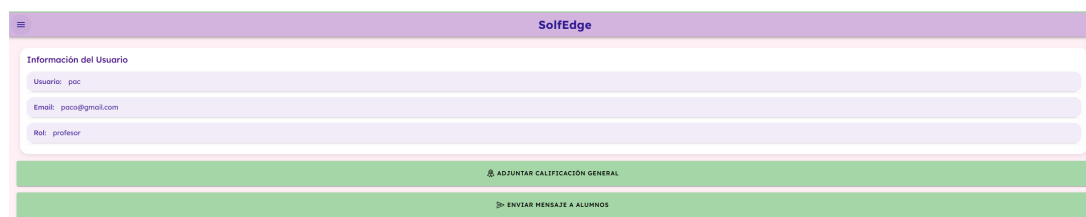


Figura 9.4: Vista del perfil de profesor

Estas imágenes reflejan la búsqueda de construir una interfaz que no solo sea visualmente atractiva, sino también usable, accesible y adaptable a los diversos entornos donde la aplicación pueda ser utilizada.

9.4 TESTING

Para garantizar la calidad, estabilidad y correcto funcionamiento del frontend, se ha implementado un conjunto de pruebas automatizadas que permiten detectar errores y verificar la integridad del código durante el ciclo de desarrollo.

9.4.1 Estrategia de pruebas

La estrategia de testing se centra principalmente en pruebas unitarias y de componentes, que validan el comportamiento individual de las unidades de código y su correcta integración en la interfaz.

- **Pruebas unitarias:** Se prueban de forma aislada funciones, métodos y componentes Angular para asegurar que cumplen con su funcionalidad esperada.
- **Pruebas de componentes:** Se comprueba la correcta renderización, interacción y comunicación con servicios de los componentes, simulando eventos y validando resultados.

9.4.2 Herramientas utilizadas

El conjunto de pruebas se ha desarrollado usando las siguientes herramientas:

- **Karma:** Ejecuta los tests en navegadores reales o headless, facilitando la integración con Angular.
- **Jasmine:** Framework de aserciones y spies para estructurar y definir las pruebas unitarias y de componentes.

Estas herramientas se integran con Angular CLI, facilitando la ejecución continua y la generación de informes de cobertura.

9.4.3 Ejemplo de prueba

A continuación se muestra un ejemplo simplificado de prueba unitaria para el componente Angular "Tab5Page" mostrado previamente, donde se comprueba que se crea correctamente y que muestra un mensaje esperado:

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { Tab5Page } from '../tab5.page';
import { ModalController, ToastController }
from '@ionic/angular/standalone';
import { provideHttpClient } from '@angular/common/http';
import { provideHttpClientTesting } from '@angular/common/http/testing';
import { of } from 'rxjs';
import { AuthService } from '../services/auth.service';
import { CalificacionService } from '../services/calificacion.service';
import { CalificacionGeneralService }
from '../services/calificacion-general.service';
import { MensajeService } from '../services/mensaje.service';
import { GrupoStateService } from '../services/grupo-state.service';
import { ActivatedRoute } from '@angular/router';

describe('Tab5Page', () => {
  let component: Tab5Page;
  let fixture: ComponentFixture<Tab5Page>;
  let modalController: ModalController;

  const authServiceMock =
  { currentUser: of({ _id: 'user123', role: 'profesor' }) };
  const calificacionServiceMock = { getCalificacionesByAlumno: jasmine.
  createSpy('getCalificacionesByAlumno').and.returnValue(of([])) };
  const calificacionGeneralServiceMock =
  { getCalificacionesByAlumnoAndGrupo: jasmine.createSpy
  ('getCalificacionesByAlumnoAndGrupo').and.returnValue(of([])) };
  const mensajeServiceMock = { crearMensaje:
  jasmine.createSpy('crearMensaje').and.returnValue(of({})) };
  const grupoStateServiceMock = { selectedGrupo$:
  of({ _id: 'grupo123', nombre: 'Test Grupo', alumnos: [] }) };

  const modalControllerMock = {
    create: jasmine.createSpy('create').and.returnValue(
      Promise.resolve({
        present: jasmine.createSpy('present').
        and.returnValue(Promise.resolve()),
        onWillDismiss: jasmine.createSpy('onWillDismiss').
        and.returnValue(Promise.resolve({ role: 'cancel' }))
      })
    ),
    dismiss: jasmine.createSpy('dismiss').and.returnValue(Promise.
    resolve())
  };
};
```



```
const toastControllerMock = {
  create: jasmine.createSpy('create').and.returnValue(
    Promise.resolve({
      present: jasmine.createSpy('present').
        and.returnValue(Promise.resolve())
    })
  )
};

const activatedRouteMock = { queryParams: of({}) };

beforeEach(async () => {
  await TestBed.configureTestingModule({
    imports: [
      Tab5Page
    ],
    providers: [
      { provide: AuthService, useValue: authServiceMock },
      { provide: CalificacionService,
        useValue: calificacionServiceMock },
      { provide: CalificacionGeneralService,
        useValue: calificacionGeneralServiceMock },
      { provide: MensajeService, useValue: mensajeServiceMock },
      { provide: GrupoStateService, useValue: grupoStateServiceMock },
      { provide: ModalController, useValue: modalControllerMock },
      { provide: ToastController, useValue: toastControllerMock },
      { provide: ActivatedRoute, useValue: activatedRouteMock },
      provideHttpClient(),
      provideHttpClientTesting()
    ],
  }).compileComponents();

  fixture = TestBed.createComponent(Tab5Page);
  component = fixture.componentInstance;
  modalController = TestBed.inject(ModalController);
  fixture.detectChanges();
});

it('should create', () => {
  expect(component).toBeTruthy();
});

it('should present mensaje modal', async () => {
  await component.presentMensajeModal();
  expect(modalController.create).toHaveBeenCalled();
});
```

```
it('should present calificacion general modal
if a group is selected', async () => {
  component.selectedGrupo = { _id: 'grupo123',
    nombre: 'Test Grupo', profesor: { _id: 'prof123' }, alumnos: [] };
  await component.presentCalificacionGeneralModal();
  expect(modalController.create).toHaveBeenCalled();
});

it('should not present calificacion general modal
if no group is selected', async () => {
  (modalController.create as jasmine.Spy).calls.reset();
  component.selectedGrupo = null;
  await component.presentCalificacionGeneralModal();
  expect(modalController.create).not.toHaveBeenCalled();
});
});
```

Este tipo de pruebas permite detectar cambios no deseados en la interfaz y lógica, facilitando el mantenimiento y evolución del frontend.

9.5 DESPLIEGUE DEL FRONTEND

El despliegue de la aplicación frontend se realiza mediante un proceso sencillo y eficiente que asegura que la versión en producción esté actualizada y optimizada para la entrega en entornos web.

9.5.1 Proceso de build y optimización

La aplicación se construye usando Angular CLI con el comando de producción, que incluye:

- **Minificación:** Se reduce el tamaño de los archivos TypeScript, CSS y HTML para mejorar tiempos de carga.
- **Tree shaking:** Se eliminan del bundle las partes de código no utilizadas, optimizando el peso final.
- **Compilación AOT (Ahead-Of-Time):** La compilación previa del código Angular permite una carga más rápida y mejor rendimiento en el navegador.

El resultado es una carpeta `www` (o `dist`) lista para ser desplegada.



9.5.2 Hosting y distribución

Se ha optado por el hosting estático gratuito y sencillo mediante **GitHub Pages**, que permite publicar la aplicación directamente desde el repositorio.

El flujo de despliegue es el siguiente:

- Se genera el build de producción localmente o en un pipeline de integración continua.
- Los archivos resultantes se suben al branch o carpeta configurada para GitHub Pages.
- GitHub Pages sirve automáticamente la aplicación desde la URL pública configurada.

Esta opción es adecuada para aplicaciones SPA (Single Page Applications) como la desarrollada, garantizando alta disponibilidad y facilidad de mantenimiento.

Este proceso asegura que el frontend esté disponible para usuarios finales de manera rápida, segura y fiable, complementando el backend con una experiencia completa y robusta.

fix backend test corregido #107

Summary

All jobs

build-frontent

build-backend

deploy-frontent

Run details

Usage

Workflow file

build-frontent

succeeded 8 hours ago in 2m 25s

- > Set up job
- > Checkout code
- > Set up Node.js
- > Install dependencies
- > Run frontend tests (Jasmine + Karma)
- > Install Ionic CLI
- > Build project
- > Upload artifacts
- > Post Set up Node.js
- > Post Checkout code
- > Complete job

198 Chrome Headless 143.0.0.0 (Linux 0.0.0): Executed 179 of 179 SUCCESS

199 TOTAL: 179 SUCCESS

200 ✓ Browser application bundle generation complete.

201 ✓ Browser application bundle generation complete.

(a) Paso de compilación del frontend en GitHub Actions

(b) Detalle de la ejecución de pruebas unitarias e integración

Figura 9.5: Proceso simplificado de despliegue en GitHub Pages.

10 INTEGRACIÓN Y DESPLIEGUE CONTINUO

Para asegurar la calidad, fiabilidad y rapidez en las entregas, se ha implementado un flujo completo de Integración Continua (CI) y Despliegue Continuo (CD) que abarca tanto el frontend como el backend del proyecto. Este pipeline automatiza la construcción, validación, pruebas y despliegue de ambas partes de la aplicación de forma coordinada.

10.1 DESCRIPCIÓN GENERAL DEL PIPELINE

El pipeline se configura para ejecutarse en cada `push` a la rama principal del repositorio, y sigue las siguientes fases:

- **Clonación y preparación:** se descarga el código fuente actualizado y se preparan los entornos de ejecución.
- **Instalación de dependencias:** se instalan todas las dependencias necesarias para frontend y backend.
- **Compilación y build:** se construyen los artefactos optimizados de frontend (aplicación Angular/Ionic) y backend (API Node.js).
- **Ejecución de pruebas automatizadas:** se ejecutan pruebas unitarias y de integración para validar la funcionalidad y estabilidad del sistema.
- **Despliegue automático:** una vez validados los builds, el backend se despliega en un entorno de producción gestionado (Render), mientras que el frontend se publica como sitio estático en GitHub Pages.

10.2 ARCHIVO DE CONFIGURACIÓN DEL PIPELINE

A continuación se muestra el fichero YML que integra ambos procesos en GitHub Actions:

```
name: Build

on:
  push:
    workflow_dispatch:
jobs:
```



```
build-frontend:
  runs-on: ubuntu-latest

  steps:
    - name: Checkout code
      uses: actions/checkout@v3

    - name: Set up Node.js
      uses: actions/setup-node@v3
      with:
        node-version: '20'
        cache: "npm"
        cache-dependency-path: ./Codigo/Frontend/package-lock.json

    - name: Install dependencies
      working-directory: ./Codigo/Frontend
      run: npm ci

    - name: Run frontend tests (Jasmine + Karma)
      working-directory: ./Codigo/Frontend
      run: npm test -- --no-watch --browsers=ChromeHeadless

    - name: Install Ionic CLI
      working-directory: ./Codigo/Frontend
      run: npm install -g @ionic/cli

    - name: Build project
      working-directory: ./Codigo/Frontend
      run: ionic build -- --base-href /RitmoApp/

    - name: Upload artifacts
      uses: actions/upload-artifact@v4
      with:
        name: www
        path: ./Codigo/Frontend/www/

build-backend:
  runs-on: ubuntu-latest

  steps:
    - name: Checkout code
      uses: actions/checkout@v3

    - name: Set up Node.js
      uses: actions/setup-node@v3
      with:
```

```
node-version: '20'
cache: "npm"
cache-dependency-path: ./Codigo/Backend/package-lock.json

- name: Install dependencies
  working-directory: ./Codigo/Backend
  run: npm ci

- name: Run backend tests (Jest)
  working-directory: ./Codigo/Backend
  run: npm test
deploy-frontend:
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  needs: build-frontend
  steps:
    - name: Download artifacts
      uses: actions/download-artifact@v4
      with:
        name: www
        path: ./www

    - name: Deploy to Github Pages
      uses: peaceiris/actions-gh-pages@v4
      with:
        github_token: ${ secrets.GH_PAT }
        publish_dir: ./www
        publish_branch: gh-pages
```

10.3 VENTAJAS DEL PIPELINE INTEGRADO

Implementar un pipeline que cubre el ciclo completo de desarrollo y despliegue aporta varios beneficios clave:

- **Coordinación entre frontend y backend:** al enlazar las tareas, se asegura que ambos se desplieguen con versiones compatibles.
- **Automatización y rapidez:** elimina tareas manuales, reduciendo errores y tiempos de entrega.
- **Control de calidad continuo:** la ejecución regular de pruebas detecta fallos tempranamente y mejora la estabilidad.
- **Trazabilidad y transparencia:** los logs y estados del pipeline facilitan la monitorización y diagnóstico de problemas.



- **Facilidad de mantenimiento:** la estructura modular del pipeline permite añadir o modificar etapas sin afectar al flujo general.

En conjunto, este sistema integrado garantiza que el proyecto se mantenga siempre actualizado, fiable y listo para su uso en producción, alineándose con las mejores prácticas modernas de desarrollo software.

11 CONCLUSIONES Y LÍNEAS DE TRABAJO FUTURO

En este capítulo se presentan las conclusiones finales del proyecto, evaluando el grado de cumplimiento de los objetivos planteados inicialmente, así como una reflexión sobre las posibles líneas de trabajo futuro que permitirían ampliar, mejorar y consolidar la solución desarrollada.

11.1 CONCLUSIONES

El objetivo principal de este proyecto consistía en resolver la problemática relacionada con la falta de recursos informáticos específicos orientados a la enseñanza del lenguaje musical. A lo largo del desarrollo de la aplicación se ha trabajado de forma progresiva para dar respuesta a esta necesidad, apoyándose en tecnologías actuales, gratuitas y ampliamente utilizadas, y poniendo el foco tanto en profesores como en alumnos.

En relación con los subobjetivos definidos inicialmente, se pueden extraer las siguientes conclusiones:

- **Entender las necesidades de profesores y alumnos de lenguaje musical:** Este objetivo se ha cubierto de forma satisfactoria mediante el análisis del contexto educativo y la definición de casos de uso reales. La aplicación incorpora funcionalidades orientadas tanto a la gestión docente (grupos, calificaciones, comunicación) como al seguimiento y participación del alumnado, reflejando una comprensión adecuada de las dinámicas habituales en la enseñanza del lenguaje musical.
- **Aprender sobre la relación existente entre la informática y el lenguaje musical:** El desarrollo del proyecto, principalmente en las fases de estudio y de investigación, ha permitido profundizar en la intersección entre ambas disciplinas, especialmente en la representación de conceptos musicales mediante herramientas digitales y en la adaptación de flujos educativos tradicionales a entornos informáticos. Este objetivo puede considerarse ampliamente cumplido, ya que ha servido como base conceptual para el diseño de la aplicación.
- **Definir una solución informática que cubra las necesidades detectadas:** Se ha definido y diseñado una solución coherente que integra backend y frontend, ofreciendo una plataforma funcional que centraliza la gestión académica y la interacción entre usuarios. La arquitectura adoptada permite escalar y extender la aplicación, lo que refuerza la validez de la solución propuesta.
- **Desarrollar la solución:** Este objetivo se ha cumplido mediante la implementación completa de la aplicación, abarcando el desarrollo del backend, el frontend híbrido y los procesos de testing

y despliegue. La aplicación es funcional, estable y usable, cumpliendo los requisitos definidos durante las fases iniciales del proyecto.

- **Llevar a cabo el proceso de desarrollo con tecnologías totalmente gratuitas:** El uso de tecnologías gratuitas y de código abierto ha sido una constante a lo largo del proyecto, tanto en el desarrollo como en el despliegue. No obstante, este enfoque también ha introducido ciertas limitaciones, especialmente en lo relativo a la implantación real en entornos productivos, lo que abre la puerta a futuras mejoras. Aun así, el objetivo puede considerarse cumplido, ya que se ha demostrado la viabilidad de una solución completa sin costes de licencias.

En conjunto, el proyecto ha logrado cumplir el objetivo principal y la mayoría de los subobjetivos planteados han sentado una base sólida para una herramienta informática orientada a la enseñanza del lenguaje musical, con margen de evolución y mejora.

11.2 LÍNEAS DE TRABAJO FUTURO

A pesar de los resultados obtenidos, existen múltiples líneas de trabajo futuro que permitirían ampliar el alcance del proyecto y mejorar su aplicabilidad en contextos reales.

11.2.1 Plan de implantación y despliegue en entornos reales

Una de las principales líneas de trabajo futuro consiste en la definición de un plan de implantación formal de la aplicación en centros educativos. El uso de tecnologías gratuitas, aunque ventajoso desde el punto de vista económico, presenta limitaciones en términos de escalabilidad, persistencia de datos y gestión avanzada de configuraciones.

Sería necesario estudiar aspectos como:

- La adaptación de los entornos de despliegue para soportar un mayor número de usuarios.
- La gestión de archivos de configuración y variables de entorno específicas para cada centro.
- La posible migración a infraestructuras más robustas manteniendo, en la medida de lo posible, el uso de herramientas gratuitas o de bajo coste.

Este trabajo permitiría acercar la aplicación a un uso real en entornos educativos.

11.2.2 Extensión de funcionalidades e innovación

Otra línea clara de evolución es la ampliación de las funcionalidades actuales, tanto para mejorar la calidad del desarrollo como para ofrecer características innovadoras y atractivas para los usuarios.

- **Integración de inteligencia artificial:** Desarrollo de pistas o ayudas automáticas en cuestionarios y ejercicios, adaptadas al progreso del alumno mediante técnicas de IA, favoreciendo un aprendizaje más personalizado.

- **Gestión masiva de usuarios mediante archivos:** Incorporación de funcionalidades que permitan dar de alta usuarios a partir de archivos con un formato definido (por ejemplo CSV), evitando la creación manual y agilizando la administración por parte del profesorado o los centros.

Estas mejoras reforzarían el carácter innovador de la aplicación y su adecuación a contextos educativos reales.

11.2.3 Acceso sin autenticación y apertura a usuarios externos

Una línea más de trabajo futura relevante consiste en extender la aplicación para ofrecer funcionalidades accesibles a usuarios no autenticados o no vinculados a ninguna entidad educativa. Esto podría incluir:

- Acceso a contenidos informativos o demostrativos sin necesidad de iniciar sesión.
- Registro de usuarios independientes, no asociados a un centro, que puedan utilizar parte de la aplicación de forma autónoma.

Esta apertura permitiría aumentar el alcance de la aplicación, facilitar su difusión y servir como punto de entrada para nuevos usuarios interesados en el aprendizaje del lenguaje musical.

En conclusión, el proyecto constituye una base sólida sobre la que construir futuras mejoras, demostrando tanto la viabilidad técnica como el potencial educativo de la solución desarrollada.

11.2.4 Creación de la aplicación móvil

Por último, aprovechando el desarrollo híbrido realizado, extender la aplicación a sistemas Android y/o iOS adaptando la aplicación donde sea necesario para su correcta ejecución en dispositivos móviles. Esto permitiría a los usuarios acceder a la plataforma desde cualquier lugar y en cualquier momento, aumentando la flexibilidad y comodidad en el uso de la aplicación.

En conclusión, el proyecto constituye una base sólida sobre la que construir futuras mejoras, demostrando tanto la viabilidad técnica como el potencial educativo de la solución desarrollada.

12 ANEXO

En este anexo se recoge información complementaria que respalda la implementación descrita a lo largo de la memoria, aportando evidencias técnicas y visuales del desarrollo realizado sin interrumpir el hilo principal del documento. Concretamente, se incluyen el acceso al código fuente del proyecto, la estructura general de los proyectos frontend y backend, así como capturas adicionales de la aplicación en funcionamiento.

12.1 REPOSITORIO DEL PROYECTO

El código fuente completo del sistema desarrollado se encuentra disponible en un repositorio público de GitHub, lo que permite consultar la implementación real del frontend y del backend, así como la evolución del proyecto durante su desarrollo.

El repositorio incluye:

- Código fuente del frontend desarrollado con Ionic y Angular.
- Código fuente del backend basado en una API REST.
- Ficheros de configuración necesarios para la ejecución y despliegue.
- Historial de commits que refleja el proceso de desarrollo.

El acceso al repositorio se realiza a través del siguiente enlace:

`https://github.com/selugc4/RitmoApp`

Este enlace permite verificar de forma directa los aspectos técnicos descritos en la memoria y facilita la reproducibilidad del proyecto en entornos de desarrollo locales.

12.2 ESTRUCTURA DEL PROYECTO

A continuación se presenta una visión general de la organización de los proyectos frontend y backend, mostrando la separación de responsabilidades y la modularidad adoptada.

12.2.1 Estructura del backend

El backend se organiza siguiendo una arquitectura modular orientada a servicios REST:

```
backend/  
|-- controllers/  
|-- models/  
|-- routes/  
|-- middleware/  
|-- services/  
`-- *.config  
`-- index.js
```

Esta disposición favorece el mantenimiento del código y la extensión futura de la aplicación, permitiendo añadir nuevas funcionalidades de forma controlada.

12.2.2 Estructura del frontend

El frontend sigue la estructura habitual de un proyecto Ionic + Angular, organizada en torno a componentes, páginas y servicios respetando así el MVC (Modelo-Vista-Controlador):

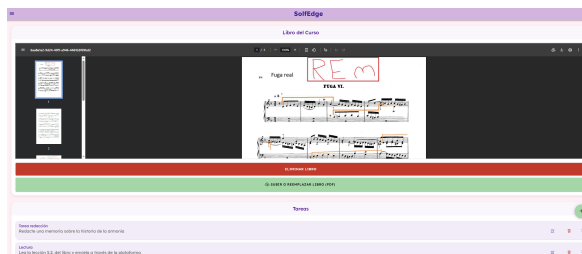
```
frontend/  
|-- src/  
|   |-- app/  
|   |-- |-- pages/  
|   |-- |-- components/  
|   |-- |-- services/  
|   |-- |-- guards/  
|   |-- |-- models/  
|   |-- `-- app-routing.module.ts  
|   |-- assets/  
|   |-- environments/  
|   |-- theme/  
|   `-- index.html  
`-- angular.json
```

Esta estructura facilita la escalabilidad del proyecto y la reutilización de componentes, manteniendo una clara separación entre lógica de presentación, lógica de negocio y navegación.

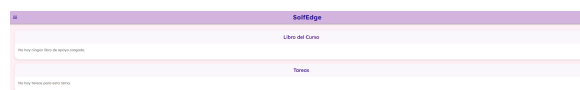
12.3 CAPTURAS ADICIONALES DE LA APLICACIÓN

En este apartado se incluyen capturas adicionales de la aplicación en funcionamiento que complementan las mostradas en el capítulo de diseño y experiencia de usuario. Estas imágenes permiten apreciar

otras vistas y flujos relevantes del sistema, reforzando la comprensión global de la aplicación desarrollada.

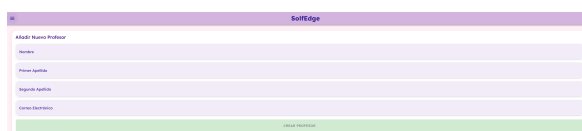


(a) Vista de profesor de rama con libro y tareas

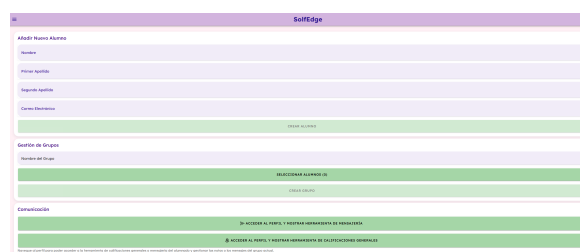


(b) Vista de alumno sin libro y sin tareas

Figura 12.1: Vista de cada rama de la aplicación



(a) Administración de administrador



(b) Administración de profesor

Figura 12.2: Vista de administración

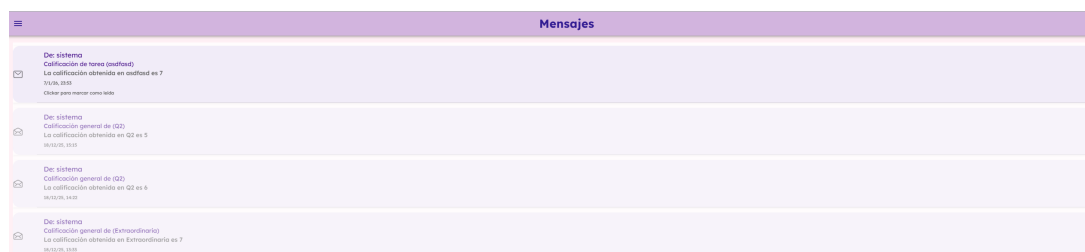


Figura 12.3: Vista de mensajería

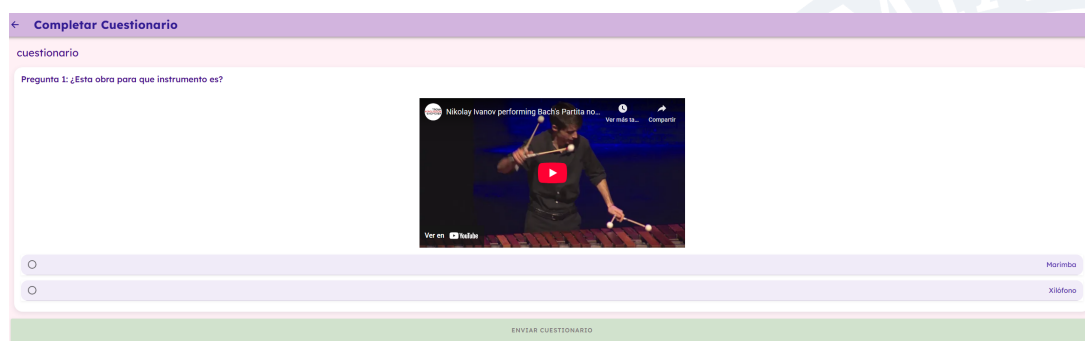


Figura 12.4: Vista de formulario para rellenar cuestionarios

Estas capturas ilustran la variedad de vistas y componentes que conforman la aplicación, evidenciando su complejidad funcional, su coherencia visual y su correcta adaptación a distintos flujos de uso.

12.4 ENTORNOS Y HERRAMIENTAS DE DESARROLLO

Además de las tecnologías principales descritas a lo largo de la memoria, durante el desarrollo del proyecto se han empleado diversas herramientas de apoyo que han facilitado la implementación, depuración y verificación del sistema. Estas herramientas no constituyen un elemento central en la justificación técnica del proyecto, pero sí han sido relevantes para el flujo de trabajo diario y la productividad durante el desarrollo.

12.4.1 Entorno de desarrollo

El desarrollo tanto del frontend como del backend se ha realizado principalmente utilizando el editor de código **Visual Studio Code**. Este entorno ha permitido trabajar de forma eficiente gracias a su ligereza, extensibilidad y buen soporte para las tecnologías empleadas en el proyecto.

Entre las extensiones más utilizadas se encuentran:

- Extensiones de soporte para **Angular** y **TypeScript** como **GeminiCLI** y **Copilot** que facilitan la navegación por el código, el autocompletado y la detección temprana de errores.
- Extensiones para **ESLint**, empleadas para mantener un estilo de código consistente y legible.
- Extensiones para **Git**, que permiten gestionar el control de versiones directamente desde el editor.

Estas herramientas han contribuido a mejorar la calidad del código y a reducir errores durante el proceso de desarrollo, sin influir directamente en la arquitectura o las decisiones técnicas del sistema.

12.4.2 Gestión y visualización de la base de datos

Para la gestión y exploración de la base de datos se ha utilizado **MongoDB Compass**, una herramienta gráfica que permite interactuar con bases de datos MongoDB de forma visual. Su uso ha sido especialmente útil durante las fases de desarrollo y depuración, permitiendo:

- Visualizar y verificar el contenido de las colecciones.
- Comprobar la estructura de los documentos almacenados.
- Realizar consultas rápidas para validar el comportamiento del backend.

Cabe destacar que MongoDB Compass se ha empleado únicamente como herramienta de apoyo, sin formar parte del sistema en producción ni afectar a la lógica de la aplicación.

12.4.3 Herramientas de apoyo adicionales

Durante el desarrollo también se han utilizado otras herramientas auxiliares, entre ellas:

- **Navegadores web con herramientas de desarrollo**, para depurar el frontend, inspeccionar el DOM y analizar el comportamiento responsive de la interfaz como Chrome que permite revisar la vista como si fuera desde móvil.
- **Terminal y gestores de paquetes** (npm), empleados para la ejecución de scripts, instalación de dependencias y lanzamiento de servidores locales.

Estas herramientas han permitido verificar el correcto funcionamiento del sistema durante las distintas fases del desarrollo, contribuyendo a un proceso de implementación más controlado y eficiente. En conjunto, el uso de estos entornos y herramientas ha facilitado el desarrollo del proyecto sin condicionar las decisiones arquitectónicas ni los objetivos planteados, sirviendo como soporte técnico durante todo el ciclo de vida de la aplicación.

BIBLIOGRAFÍA

- [1] Universidad Europea, “Lenguaje musical: qué es, cómo enseñarlo y fichas útiles,” Aug. 2025. [Online]. Available: <https://universidadeuropea.com/blog/lenguaje-musical/>
- [2] Estudio Música, “Estudio música.” [Online]. Available: <https://estudiomusica.com/>
- [3] I. Rodríguez Heras, “El lenguaje musical a través de las TIC en Educación Primaria,” 2022. [Online]. Available: <https://uvadoc.uva.es/bitstream/handle/10324/56944/TFG-G5671.pdf>
- [4] I. Gorbunova and H. Hiner, “Music Computer Technologies and Interactive Systems of Education in Digital Age School,” in *Proceedings of the International Conference Communicative Strategies of Information Society (CSIS 2018)*, 2019. [Online]. Available: <https://www.atlantis-press.com/proceedings/csis-18/55913802>
- [5] M. García Aguirre, “Aplicación informática para la introducción del lenguaje musical (LeeMúsica) en dispositivos móviles. Niveles 1 y 2 (3 años),” 2017. [Online]. Available: <https://uvadoc.uva.es/bitstream/10324/22384/1/TFG-G2315.pdf>
- [6] S. Marić, “Online gaming to learn music and English language in music and ballet school solfeggio education,” *Hellenic Journal of Music, Education and Culture*, vol. 6, no. 2, p. 54, 2015. [Online]. Available: <https://hejmec.eu/journal/index.php/HeJMEC/article/view/58/54>
- [7] L. Parshina, “Multimedia technologies as a tool for teaching supervision over the students’ skills (within the course on “music theory training”),” *Life Science Journal*, vol. 11, no. 6, pp. 547–551, 2014. [Online]. Available: https://www.lifesciencesite.com/ljsj/life1106/081_24754life110614_547_551.pdf
- [8] Ionic Team, “Ionic Framework Documentation.” [Online]. Available: <https://ionicframework.com/docs>
- [9] Angular Team, “Angular Documentation.” [Online]. Available: <https://angular.dev/overview>
- [10] ESIC Business & Marketing School, “Modelo-Vista-Controlador: qué es, cómo funciona y ventajas,” 2025. [Online]. Available: <https://www.esic.edu/rethink/tecnologia/modelo-vista-controlador-que-es-como-funciona-y-ventajas-c>
- [11] Jasmine Team, “Jasmine Testing Framework Documentation.” [Online]. Available: <https://jasmine.github.io/>
- [12] Karma Core Team, “Karma Testing Documentation.” [Online]. Available: <https://karma-runner.github.io/6.4/index.html>

- [13] Node.js Foundation, “Node.js Documentation.” [Online]. Available: <https://nodejs.org/docs/latest/api/>
- [14] Express.js, “Express Framework Documentation.” [Online]. Available: <https://expressjs.com/es/>
- [15] JWT.io, “Introduction to JSON Web Tokens.” [Online]. Available: <https://www.jwt.io/introduction#what-is-json-web-token>
- [16] SmartBear, “Swagger Documentation.” [Online]. Available: <https://swagger.io/docs/>
- [17] Jest Team, “Jest Documentation.” [Online]. Available: <https://jestjs.io/docs/getting-started>
- [18] npm, Inc., “Supertest Package.” [Online]. Available: <https://www.npmjs.com/package/supertest>
- [19] MongoDB, Inc., “MongoDB Documentation.” [Online]. Available: <https://www.mongodb.com/docs/>
- [20] Mailjet, “Mailjet Email Guides.” [Online]. Available: <https://dev.mailjet.com/email/guides/>
- [21] yavisht, “GenratrAPI Repository.” [Online]. Available: <https://github.com/yavisht/GenratrAPI>
- [22] GitHub, “GitHub Documentation.” [Online]. Available: <https://docs.github.com/es>
- [23] Render, “Render Documentation.” [Online]. Available: <https://render.com/docs>



El proyecto ha consistido en el estudio e investigación de las diferentes herramientas, tecnologías o cualquier aspecto que pudiera relacionar la asignatura de lenguaje musical con la ingeniería informática, así como la definición, desarrollo y despliegue de una solución informática para resolver la carencia de recursos informáticos en el proceso de aprendizaje del lenguaje musical.

The project has consisted of the study and research of different tools, technologies, and any other aspects that could relate the subject of music theory to computer engineering, as well as the definition, development, and deployment of a software solution aimed at addressing the lack of digital resources in the process of learning music theory.